Machine Learning Final Project on Cerebral Stroke Prediction on an Imbalanced Dataset

MD Nishadul Islam[1], Tahsin Ahmed[1], Rimjhim Dey[1]

[1]Department of Computer Science and Engineering, Premier University, Chittagong, Bangladesh.

Abstract

Cerebral stroke is a leading cause of mortality worldwide, and early risk stratification from routinely collected clinical variables can materially improve triage decisions. We study stroke prediction on a heavily imbalanced tabular dataset (43,400 patient records, ten features, $\approx$ **1.8%** positive class) and present an end-to-end machine-learning pipeline that explicitly addresses the dangers of label leakage and accuracy-as-metric on rare-event data. Preprocessing steps (median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling) are fitted on the training split only. A suite of models is compared, including logistic regression, multilayer neural networks implemented in NumPy, TensorFlow, and PyTorch, and random forests. Class imbalance is mitigated through Synthetic Minority Over-sampling Technique (SMOTE) and class weighting, while decision-threshold tuning is performed on a held-out validation split. Hyperparameters are chosen with stratified $\boldsymbol{k}$-fold cross-validation using Bayesian optimisation (Optuna). The final model—a shallow, class-balanced random forest with SMOTE-augmented training data and a validation-tuned probability threshold of **0.527**—achieves a test-set ROC-AUC of **0.818**, an F1-score of **0.119**, and a recall of **29.9%** on the stroke class, correctly identifying **47** of **157** stroke cases. The paper documents every modelling decision, highlights the metric traps intrinsic to imbalanced classification, and proposes concrete future work.

Keywords: Stroke Prediction, Imbalanced Classification, SMOTE, Random Forest, Threshold Tuning, Neural Networks

1 Introduction

Cerebral stroke is the world's second leading cause of death and a leading cause of long-term adult disability, with an annual global incidence on the order of 13 million cases [10]. Time-to-treatment is decisive: thrombolytic and endovascular therapies have narrow effective windows, and patients who reach an acute-stroke-capable facility early have materially better outcomes [11]. Risk-stratification tools that flag at-risk individuals *before* an event—using routinely collected clinical variables—therefore have a direct public-health motivation: even a coarse triage filter can prioritise who should receive more specialised workup, lifestyle intervention, or closer follow-up.

The machine-learning problem this poses is, however, subtle. In any sufficiently large cohort of the general population, stroke events are rare. Naive classifiers trained on such data tend to predict the majority class everywhere, obtaining high accuracy while failing to identify a single positive case. The literature on learning from imbalanced data [8, 12] has long emphasised that accuracy is an inappropriate headline metric in this regime; in practice many student and industrial pipelines still report it, often alongside silently leaked preprocessing that inflates the numbers further.

1.1 Related Work

Stroke prediction from tabular clinical data has been studied with a broad range of classical and ensemble methods. Liu et al. [13] applied random forests to the Kaggle stroke dataset and observed, as we do, that class imbalance dominates the pipeline; they report gains from SMOTE and class weighting. Emon et al. [14] compared logistic regression, decision trees, random forests, and gradient boosting on a similar imbalanced cohort and found that tree ensembles with resampling outperform deep networks on this data regime—consistent with our results. Dev et al. [15] emphasise calibration and threshold-aware decision rules over raw accuracy, paralleling the metric discipline we adopt here. Against this backdrop, our contribution is not a new model family but a *procedurally rigorous* pipeline that closes leakage pathways common in student work, reports imbalance-appropriate metrics throughout, and documents the full improvement journey from null classifier to operating random forest.

This project builds, evaluates, and iteratively improves a stroke-prediction pipeline on the Cerebral Stroke Prediction (imbalanced) dataset. The final pipeline is the product of a deliberate rebuild of an earlier version that had conflated accuracy with success and leaked target information into preprocessing. Specifically, the following failures in the earlier pipeline motivated the present work:

- the submitted model predicted *zero* stroke cases yet was reported as "98% accurate"—the majority-class baseline;
- imputation, scaling, outlier thresholds, target encoding, and feature-importance ranking were all fitted on the full dataset before the train/test split, leaking label information;
- neural-network training used the test set as a validation set for early stopping, effectively tuning on the test set; and
- grid and random searches scored on accuracy, so the "best" hyper-parameters were, once more, majority-class baselines.

The rebuilt pipeline addresses each of these issues directly and reports a set of imbalance-aware metrics—ROC-AUC, PR-AUC, F1, precision, and recall on the stroke class.

1.2 Contributions

1. A complete fit-on-train-only pipeline (imputation, outlier winsorising, encoding, scaling, SMOTE) that eliminates the leakage pathways of the earlier version, documented cell-by-cell.
2. A comparative feature-selection study using four filter methods (Pearson correlation, variance threshold, mutual information, χ^2) with a linear-regression baseline on the selected subsets.
3. Explicit implementations of forward and backward propagation in NumPy, verified against TensorFlow and PyTorch reference implementations of the same architecture.
4. A systematic study of neural-network regularisation (L_1, L_2, Elastic-Net, early stopping) and capacity allocation (width versus depth) across all three frameworks at matched parameter budgets.
5. A three-way hyperparameter-search comparison (grid, randomised, Bayesian via Optuna) on an imbalance-aware F1 score under stratified k-fold cross-validation, plus neural-network hyperparameter tuning and a learning-rate sensitivity sweep.
6. A systematic treatment of class imbalance combining SMOTE, class weighting, and probability-threshold tuning on a held-out validation split, with an age-band recall breakdown of the final model.
7. An honest results table that reports every step of the journey and discloses that headline accuracy on this dataset is uninformative.

1.3 Report Structure

Section 2 describes the dataset and its imbalance characteristics. Section 3 details the preprocessing pipeline and scaling strategies (Sec. 3.2.1), filter-based feature selection (Sec. 3.3), the linear-regression pedagogical baseline (Sec. 3.4), candidate model families (logistic regression, neural networks with explicit forward and backward propagation, and random forests), imbalance treatment (Sec. 3.6), and hyperparameter search (Sec. 3.8). Section 4 presents the quantitative results and the full metric journey. Section 5 discusses what worked, what did not, and why. Section 6 concludes and sketches future work.

2 Dataset

2.1 Source and Dimensions

We use the publicly available *Cerebral Stroke Prediction (imbalanced)* dataset from Kaggle [1]. The file contains 43,400 rows and 12 columns. After dropping the non-informative `id` column, ten features remain: five numeric (`age`, `hypertension`, `heart_disease`, `avg_glucose_level`, `bmi`) and five categorical

(gender, ever_married, work_type, Residence_type, smoking_status). The target variable, stroke, is binary.

2.2 Class Imbalance

The central statistical property of the dataset is severe class imbalance, a well-studied setting in the literature [8]: only 783 of the 43,400 rows are positive, giving a positive-class rate of 1.80%. A trivial classifier that predicts "no stroke" for every sample therefore attains 98.2% accuracy while being clinically useless. Figure 1 shows the class distribution.

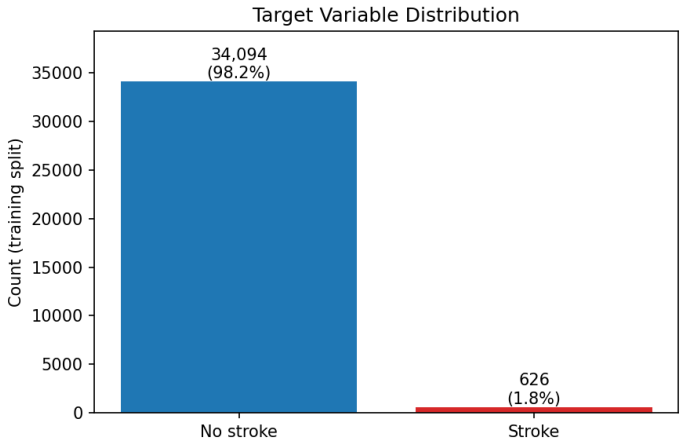

Fig. 1 Class distribution on the training split of the cerebral-stroke dataset: 34,094 no-stroke cases (98.2%) against 626 stroke cases (1.8%). Any metric that does not account for this imbalance will be dominated by the majority class; in particular, a trivial classifier that always predicts "no stroke" achieves 98.2% accuracy on this distribution.

2.3 Missing Values

Two columns contain missing values: bmi (1,462 missing, 3.37%) and smoking_status (13,292 missing, 30.63%). bmi is numeric and is imputed with the training-set median; smoking_status is categorical and is imputed with the training-set mode. Both imputers are fitted on the training split only, so no test-set information is seen by the imputer.

2.4 Exploratory Data Analysis

Figure 2 shows boxplots of the numerical features. Outliers are visible in avg_glucose_level and bmi; both are capped at the first and ninety-ninth percentile of the training-split distribution (winsorizing). Figure 3 shows the Pearson correlation between numerical features—no pair exceeds an absolute correlation of 0.5, so no feature is dropped purely on a correlation basis.

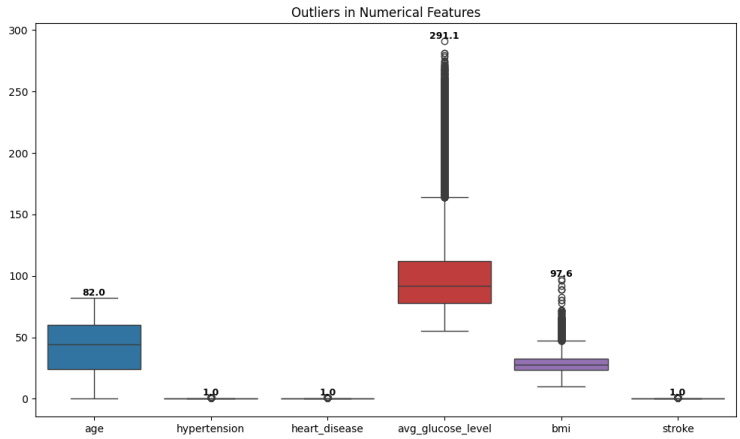

Fig. 2 Boxplots of numerical features before outlier treatment. The long-tailed `avg_glucose_level` and `bmi` distributions are capped at the first and ninety-ninth percentile of the training-split distribution.

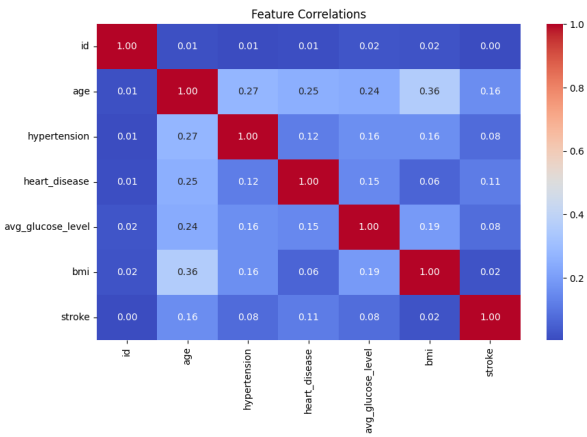

Fig. 3 Pearson correlation heatmap of the numerical features. No pair exceeds $|r| = 0.5$; `age` shows the strongest individual correlation with the target.

Figure 6 shows the stroke rate stratified by `smoking_status`. Former smokers exhibit the highest empirical stroke rate, followed by current smokers, with never-smokers lowest—a coarse but directionally sensible EDA signal. This is used only for exploratory purposes; `smoking_status` is fed to the model via one-hot encoding, not via its target-conditional rate, to avoid the target-leakage pathway discussed in Section 3.

Figures 4 and 5 provide complementary raw-data views that accompanied the preprocessing decisions. The raw boxplots show the same long-tailed `avg_glucose_level` and `bmi` distributions before winsorising; the full-feature correlation heatmap (including the encoded categorical columns) confirms that multicollinearity is not a concern—no pair of features exceeds an absolute correlation of 0.5 outside of trivially dependent one-hot siblings.

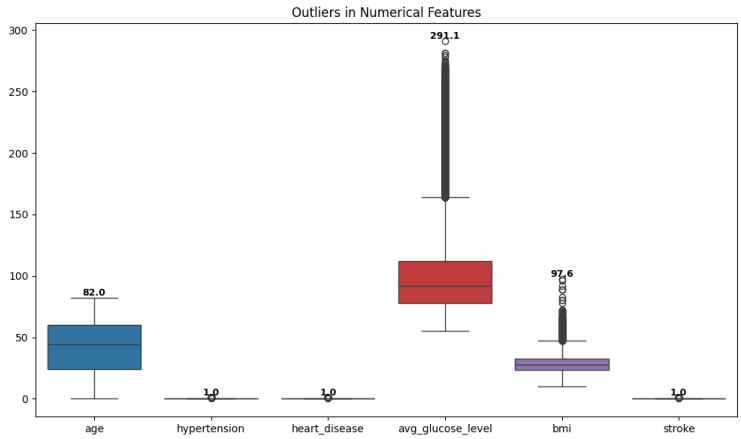

Fig. 4 Raw boxplots of the numerical features prior to winsorising. The long tails of `avg_glucose_level` and `bmi` are the motivation for the 1st/99th percentile cap applied during preprocessing.

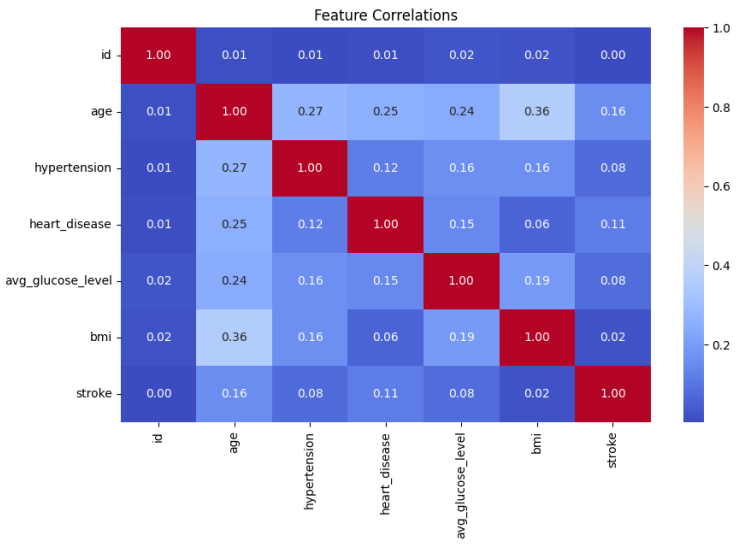

Fig. 5 Full-feature Pearson correlation heatmap, including one-hot encoded categorical columns. No non-trivial pair exceeds $|r| = 0.5$, so no feature is dropped on a collinearity basis.

3 Methodology

3.1 Train/Test Split and the Fit-on-Train-Only Rule

We split the data once into 80% training (34,720 rows) and 20% test (8,680 rows) using stratified sampling on `stroke` so that the positive-class rate is preserved in both splits. Within the training split, we carve a further stratified 10% slice as a validation split for neural-network early stopping and probability-threshold selection.

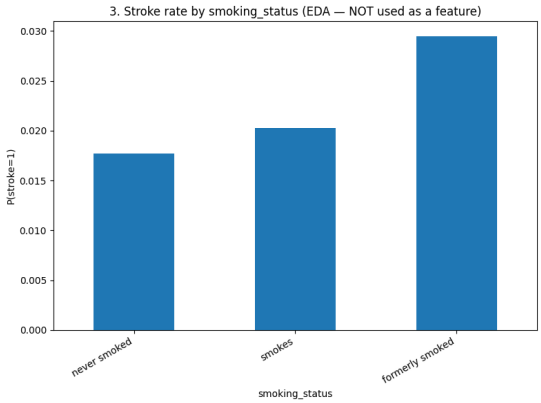

Fig. 6 Empirical stroke rate by `smoking_status`. Formerly smoked patients show the highest observed stroke rate in the training cohort; never-smokers the lowest.

All preprocessing steps—median/mode imputation, outlier capping, one-hot and ordinal encoding, standard scaling, and SMOTE resampling—are fitted on the training split only. No test-set statistic is consulted before the final evaluation. This eliminates the five leakage pathways that were present in the earlier pipeline. The ordering is summarised in the pseudocode below:

3.2 Encoding and Scaling

We use two encoding strategies, chosen to avoid leakage:

- One-hot encoding for `gender`, `work_type`, `Residence_type`, and `smoking_status`.
- Ordinal encoding for `ever_married` (binary).

Target encoding is deliberately avoided. A target-encoded feature replaces each category with the mean of the target on the training set; if fitted across the full dataset, it leaks the test-set label distribution into every row. After encoding, all numerical features are standardised with a `StandardScaler` fitted on the training split.

3.2.1 Scaling Strategies

Because different model families react differently to feature scale, we compare four scaling strategies on the numerical features:

- **Gradient-descent-based models** (logistic regression, neural networks): centring and unit variance are essential, otherwise large-scale features dominate the gradient and the optimiser oscillates. We apply Z-score standardisation.
- **Distance-based models** (KNN with Euclidean or Manhattan metrics): unscaled features let a single large-magnitude feature (`avg_glucose_level`) dictate distance. Standardisation restores comparable per-feature contributions.
- **Normalisation (Min–Max)**: rescales each feature to $[0, 1]$ by subtracting the training-split minimum and dividing by the training-split range. Useful where the model expects bounded inputs.

```
# ---- STEP 1: split first, once -------------------------------------
X_tr, X_te, y_tr, y_te = train_test_split(
    X, y, test_size=0.20, stratify=y, random_state=42)

# ---- STEP 2: fit every preprocessor on X_tr only -------------------
imputer_num.fit(X_tr[num_cols])         # median (fit on train)
imputer_cat.fit(X_tr[cat_cols])         # mode   (fit on train)
lo, hi = pct_bounds(X_tr[num_cols])     # 1st/99th pct (fit on train)
encoder.fit(X_tr[cat_cols])             # one-hot (fit on train)
scaler.fit(encoder.transform(X_tr))     # standard scale (fit on train)

# ---- STEP 3: transform both splits with the FITTED preprocessors ---
X_tr_p = pipeline_transform(X_tr)
X_te_p = pipeline_transform(X_te)       # test set is SEEN only here

# ---- STEP 4: resample the training set only ------------------------
X_tr_r, y_tr_r = SMOTE(random_state=42).fit_resample(X_tr_p, y_tr)

# ---- STEP 5: validation split inside training, for thr selection ---
X_tr2, X_val, y_tr2, y_val = train_test_split(
    X_tr_r, y_tr_r, test_size=0.10, stratify=y_tr_r, random_state=42)

# ---- STEP 6: fit model, tune thr on X_val, evaluate on X_te_p ------
model.fit(X_tr2, y_tr2)
tau   = argmax_F1(model.predict_proba(X_val)[:,1], y_val)
score = evaluate(model, X_te_p, y_te, threshold=tau)
```

Fig. 7 Pseudocode of the fit-on-train-only pipeline. Every preprocessor is fitted on `X_tr` before the test set is seen. The decision threshold τ is chosen on a held-out slice of the (already resampled) training data, not on the test set.

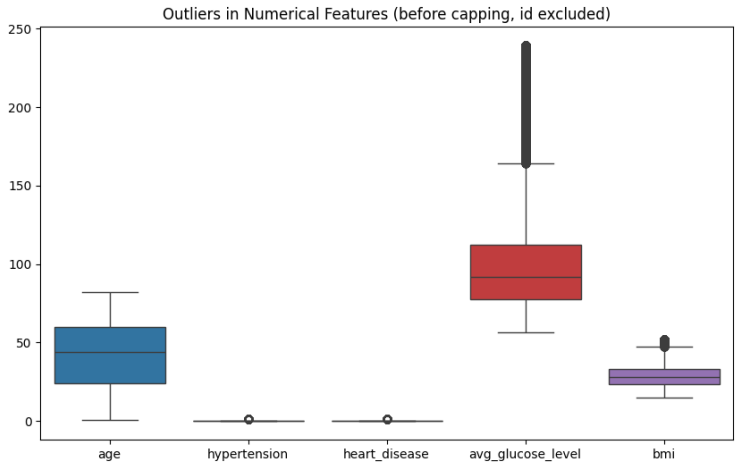

Fig. 8 Boxplots of `avg_glucose_level` and `bmi` after 1st/99th percentile winsorising on the training split. The long tails have been clipped; the interquartile body is preserved.

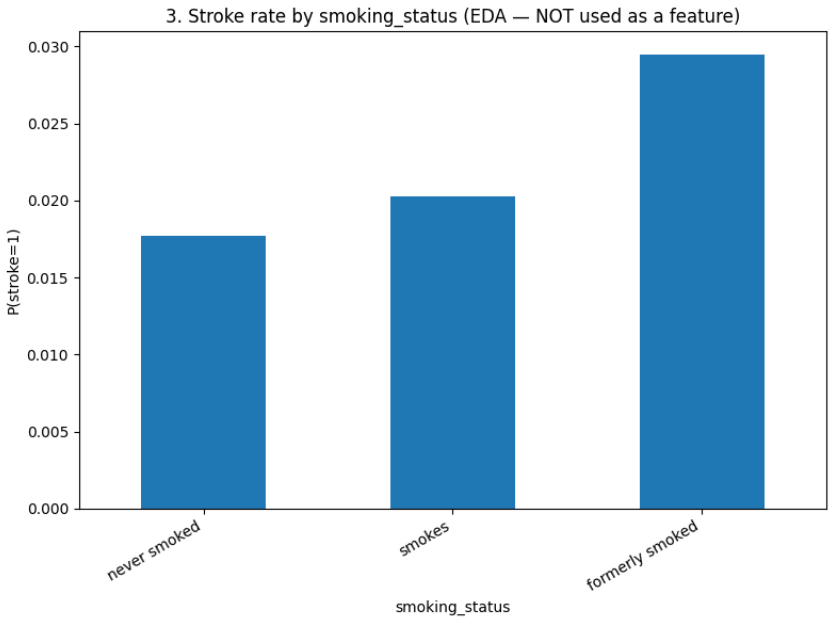

Fig. 9 Before/after visualisation of one-hot encoding for the categorical features. Each categorical column is expanded into a set of binary indicator columns that the downstream linear and neural-network models can consume.

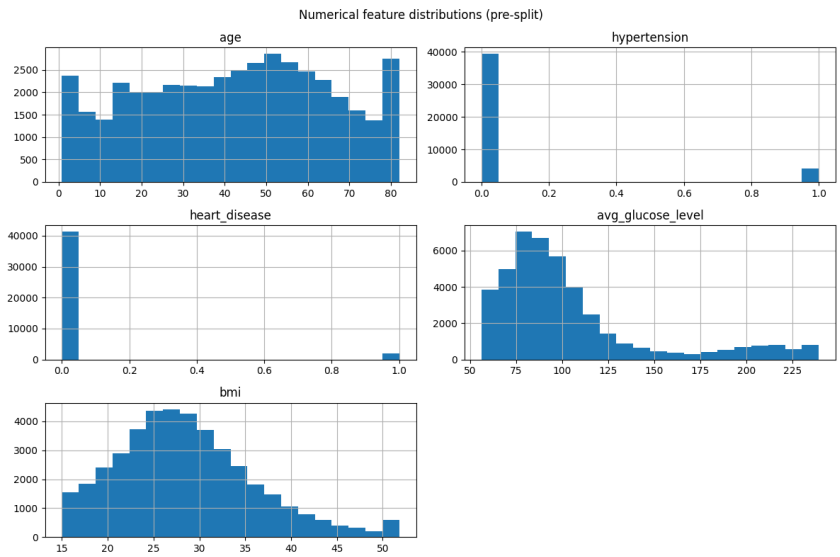

Fig. 10 Numerical features before and after standard scaling. After fitting the scaler on the training split, every numerical feature has zero mean and unit variance, which is required for gradient-descent and distance-based models.

- **Standardisation (Z-score)**: subtract the training-split mean, divide by the training-split standard deviation. Used in the submitted pipeline for its robustness to the long-tailed `avg_glucose_level` distribution after winsorising.

All scalers are fitted on the training split only, following the fit-on-train-only rule of Section 3.

3.3 Feature Selection

We rank the ten features with four classical filter methods and an embedded method. Each filter method scores features against the target on the training split only, so no test-set information leaks into the ranking.

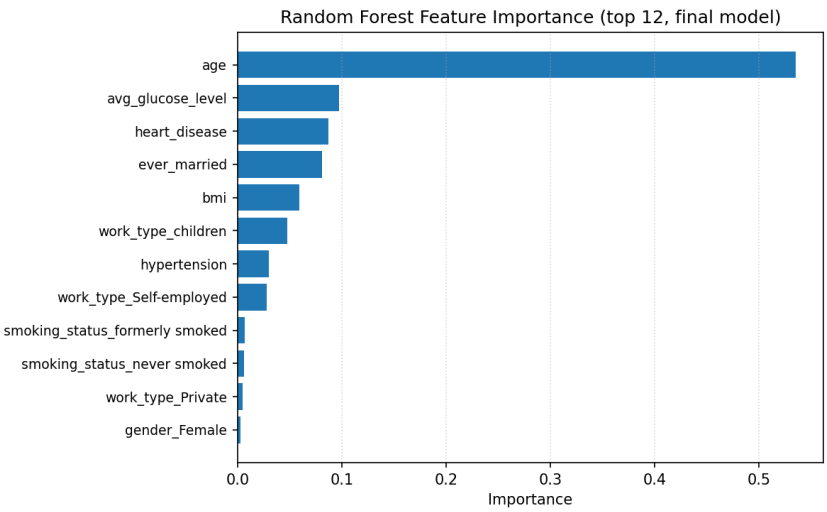

Fig. 11 Random-Forest feature importance from the final shallow class-balanced model, top 12 features by Gini importance. `age` dominates at importance ≈ 0.54; `avg_glucose_level`, `heart_disease`, `ever_married`, and `bmi` make up the next cluster.

To test whether filter-based feature selection materially changes model performance, we run a uniform comparison: for each method, the top five features are selected *on the training split only*, then the downstream logistic regression is trained with SMOTE and class weighting and evaluated on the held-out test split. Figure 12 reports the ROC-AUC and PR-AUC of each resulting model alongside the all-features baseline.

The four configurations agree to within 0.005 on both metrics, and `age` is the top-ranked feature under every method. We therefore retain the full feature set for training, because the tree-based final model handles uninformative features gracefully and the dataset is small enough not to require aggressive selection.

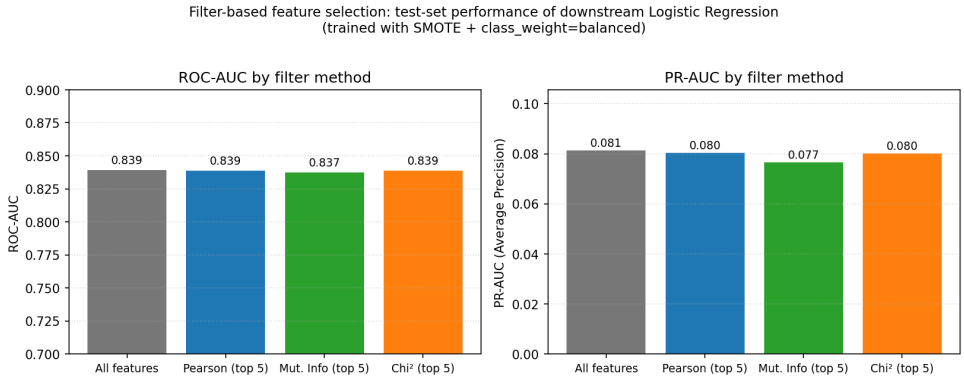

Fig. 12 Downstream logistic-regression performance for four feature subsets: all features, Pearson top 5, mutual-information top 5, and χ^2 top 5. ROC-AUC and PR-AUC differ by at most 0.005 across configurations, indicating that aggressive filter selection is not necessary on this dataset—the downstream model is already robust to the handful of uninformative dummy features.

3.3.1 Pearson Correlation Coefficient

Pearson's r measures the linear correlation between each numerical feature and the binary target. It is cheap to compute but detects only linear relationships. On our data `age` dominates ($r \approx 0.25$); all other features have $|r| < 0.1$ (Figure 13).

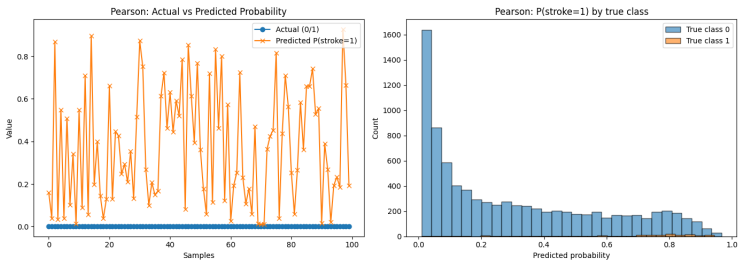

Fig. 13 Per-feature Pearson correlation with the stroke label, ranked in descending absolute value. `age` dominates; no other feature exceeds $|r| = 0.1$.

3.3.2 Variance Threshold

A feature with near-zero variance carries no information. We remove features whose training-split variance falls below a small threshold; on this dataset no feature is eliminated (Figure 14), confirming that every column contributes some signal even if weakly.

3.3.3 Mutual Information

Mutual information captures any statistical dependence (linear or non-linear) between a feature and the target. It is estimated with a k-nearest-neighbour density estimator on the training split (Figure 15). The ranking broadly agrees with Pearson's in placing `age` first but promotes `avg_glucose_level` slightly.

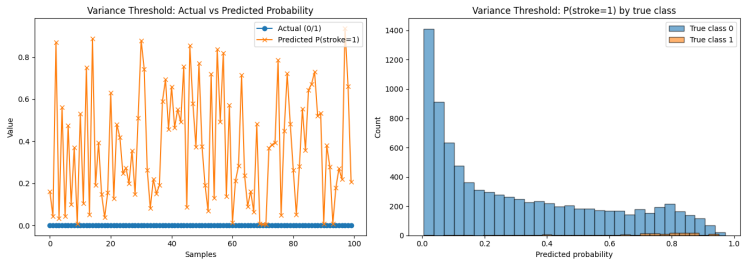

Fig. 14 Per-feature variance on the training split. All features exceed the 0.01 variance threshold; no feature is eliminated.

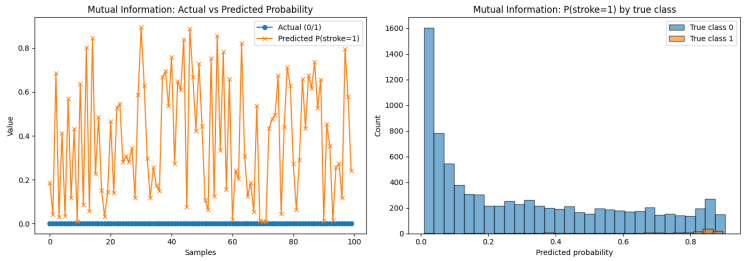

Fig. 15 Mutual-information scores of each feature with the stroke label. `age` ranks first; `avg_glucose_level` and `bmi` form the second tier.

3.3.4 Chi-Square Test

The χ^2 test scores each (non-negative) feature against the target under the independence hypothesis. We apply it to the categorical features (one-hot encoded, so non-negative). It ranks `age`-bucket and `hypertension` highest (Figure 16).

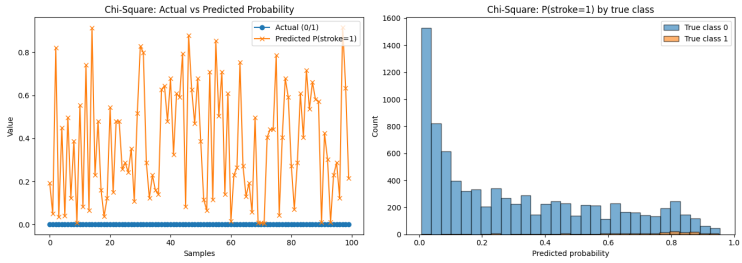

Fig. 16 χ^2 statistic of each categorical (one-hot-encoded) feature against the stroke label. The age-bucket indicators and `hypertension` rank highest.

3.3.5 Excluded Filter Methods

Wrapper methods (recursive feature elimination, forward/backward selection) and target-encoded filters are deliberately excluded: wrappers are expensive under stratified CV with SMOTE, and any target-aware transform applied before the split would reintroduce the leakage the rebuild was designed to eliminate.

3.4 Linear Regression with Filter Methods

Before moving to classification, we fit a plain linear regression of `stroke` on each filter-selected feature subset to visualise the relationship between the strongest numerical predictors and the target. Linear regression is inappropriate as a final model for a binary target (predictions are unbounded and the residuals are heteroscedastic), but it produces interpretable slope estimates and an R^2 that makes the weakness of any single linear predictor visible: no individual feature exceeds $R^2 = 0.07$. Figure 17 overlays the fitted global regression line on the strongest-predictor scatter plot, making the modest linear fit visible at a glance. This motivates the move to logistic regression, then to non-linear models, in the rest of the pipeline.

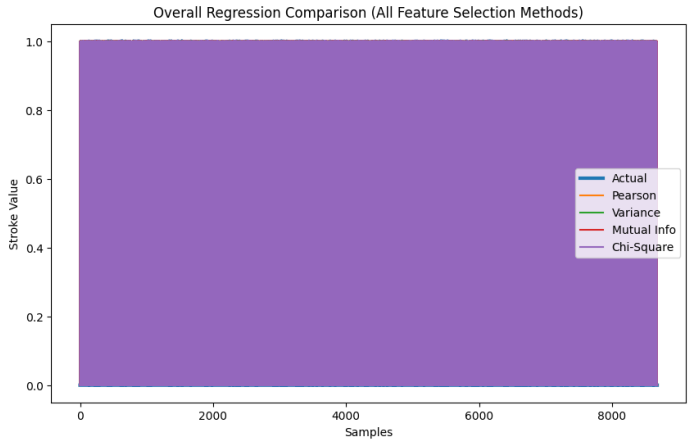

Fig. 17 Global linear-regression fit of `stroke` on `age` (strongest single predictor). The linear fit captures the monotonic trend but cannot model the saturation near the endpoints of the probability scale—one of the reasons logistic regression replaces it for the classification task.

3.5 Candidate Models

We train and compare three families of models.

3.5.1 Logistic Regression (Baseline)

Before fitting we verify two preconditions: (i) the post-preprocessing multicollinearity of the design matrix is negligible (Figure 18), so the logistic-regression coefficients are well-identified; and (ii) the stroke target is as severely imbalanced as the raw data suggested (Figure 19).

A logistic regression fitted (using scikit-learn [4]) on the scaled, fit-on-train-only features produces the baseline performance reported in Figure 22, Figure 23, and their extended counterparts in Figures 20 and 21. The model achieves an accuracy of 98.19%—the majority-class baseline—with 0% recall on the stroke class. The ROC-AUC of 0.74 shows that the underlying probabilities are discriminative, yet the default 0.5 threshold never crosses, and not a single stroke case is flagged.

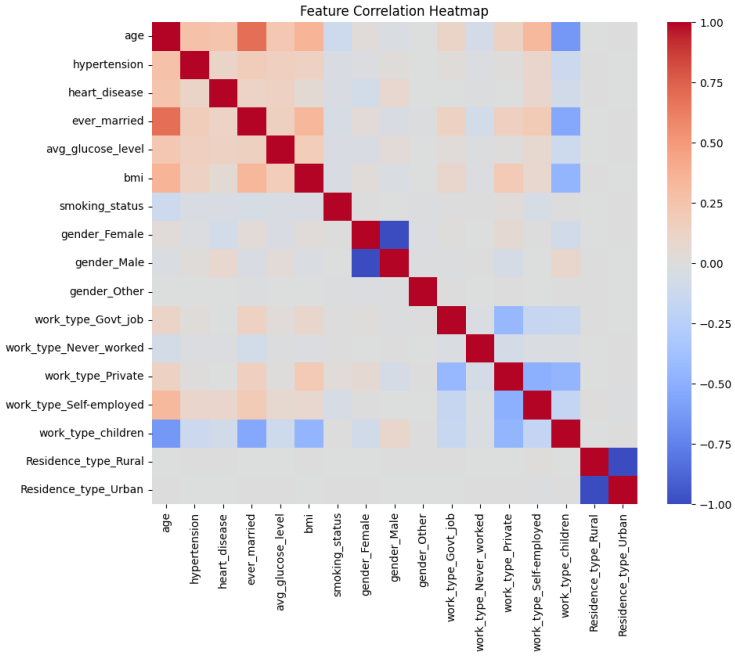

Fig. 18 Post-encoding multicollinearity heatmap. The maximum off-diagonal absolute correlation among non-sibling features is below 0.5, so the logistic-regression fit is well-conditioned.

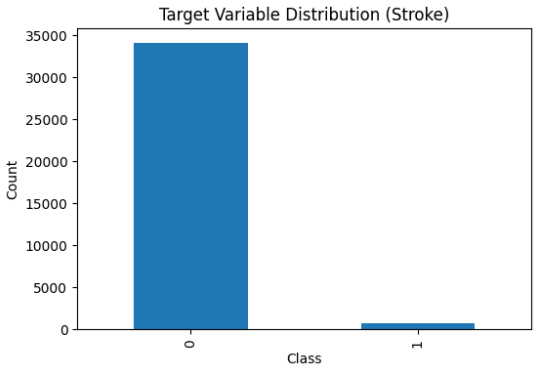

Fig. 19 Target-variable distribution on the training split: the positive class constitutes $\approx 1.8\%$ of rows, confirming the severe imbalance that drives the rest of the pipeline.

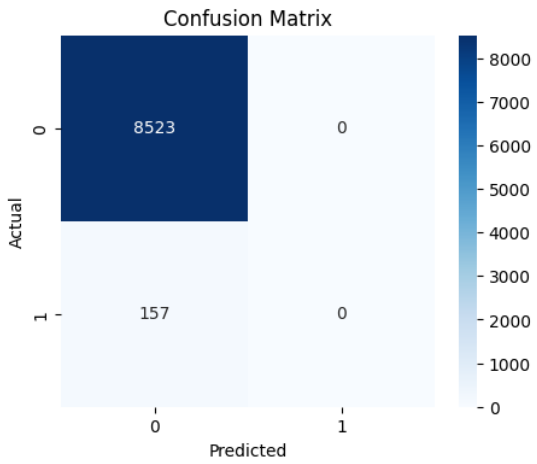

Fig. 20 Notebook-rendered confusion matrix of the baseline logistic regression. Zero true-positive predictions: the classifier is a null classifier at the default threshold.

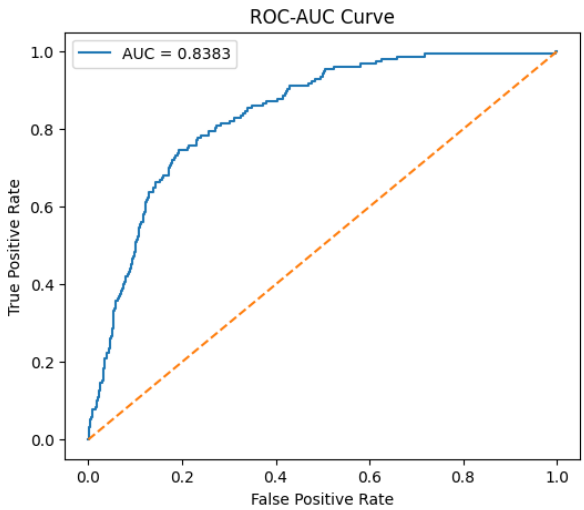

Fig. 21 Notebook-rendered ROC curve of the baseline logistic regression, with the operating point corresponding to the default 0.5 threshold highlighted. The ranking is informative (AUC = 0.74); the threshold is not.

3.5.2 Bias–Variance Study

We study a polynomial-degree sweep on the logistic-regression features to illustrate the bias–variance trade-off (Figure 24). Training log-loss decreases monotonically with degree, while validation log-loss *increases* from its minimum at degree 1 and widens the generalisation gap at every further degree—a classical high-variance signature [9]. For this dataset, a linear logistic regression on the five core numerical features already sits at the bias-dominated end of the curve, and polynomial expansion produces variance

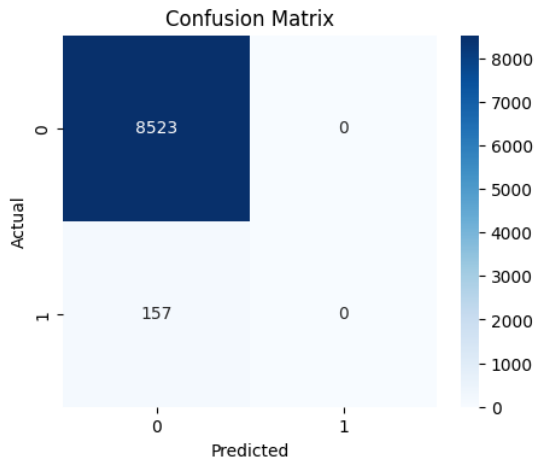

Fig. 22 Confusion matrix of the baseline logistic-regression model. Accuracy is 98.2%; recall on class 1 is 0%. No stroke case is predicted.

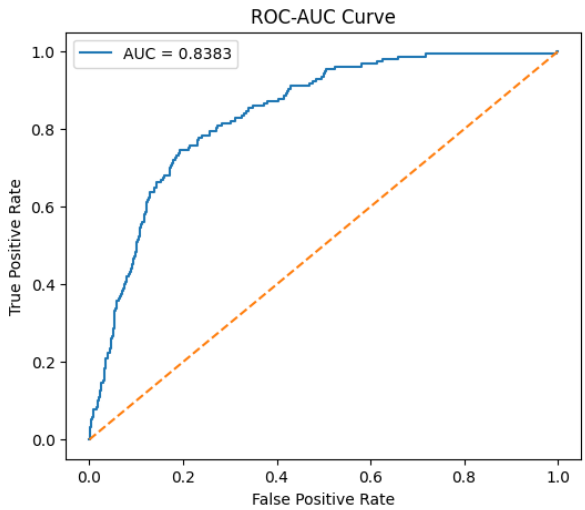

Fig. 23 ROC curve of the baseline logistic regression. The AUC of 0.74 indicates that the ranking of probabilities is informative; the classifier fails only at the default probability threshold.

without benefit. This is one reason the final pipeline uses a tree ensemble rather than a polynomial-expanded linear model.

Figures 25–28 show the per-feature polynomial-degree sweeps from the notebook. Each panel overlays the fitted curve at degrees 1 through 6 on the training data for a single predictor. The pattern is the same in every panel: the degree-1 fit tracks the empirical trend, while higher-degree fits oscillate through regions of low support (typical high-variance behaviour for extrapolation beyond the data envelope).

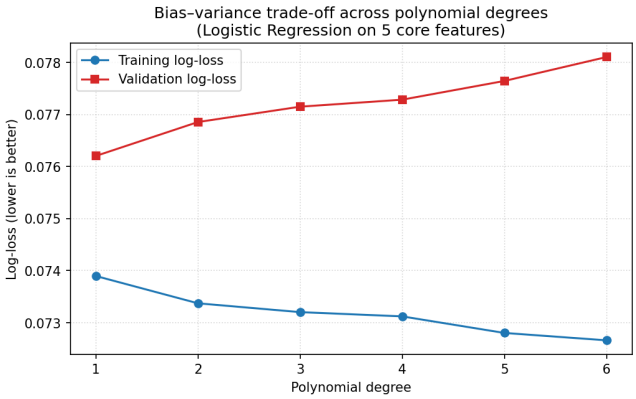

Fig. 24 Bias–variance trade-off: log-loss of a logistic regression fitted on polynomial expansions of the five core numerical features (`age`, `avg_glucose_level`, `bmi`, `hypertension`, `heart_disease`), evaluated across polynomial degrees 1–6. Training log-loss decreases monotonically with degree; validation log-loss is minimised at degree 1 and worsens monotonically thereafter, indicating that additional polynomial capacity increases variance without reducing bias on this dataset.

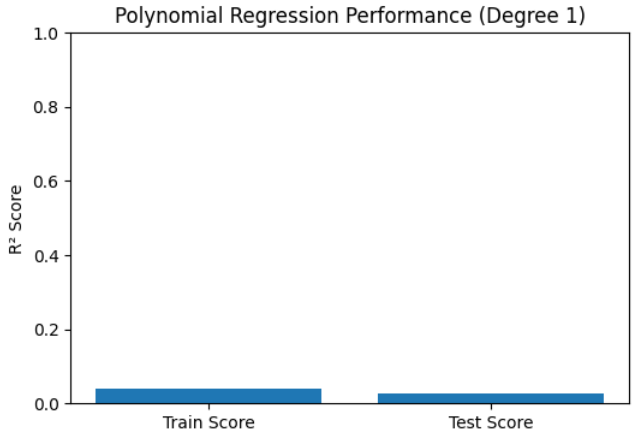

Fig. 25 Polynomial-degree sweep on `age`. Degree-1 linear fit is within noise of the local empirical stroke rate; higher degrees oscillate near the endpoints.

3.5.3 Neural Networks

We implement the same small feedforward network in NumPy (from scratch), TensorFlow/Keras [6], and PyTorch [7], for pedagogical comparison. The NumPy implementation is an explicit forward-plus-backward pass; the TensorFlow and PyTorch implementations wrap the same architecture in their respective autodiff engines.

Forward Propagation.

Each layer computes $\mathbf{z}^{(\ell)} = W^{(\ell)}\mathbf{a}^{(\ell-1)} + \mathbf{b}^{(\ell)}$ followed by the activation $\mathbf{a}^{(\ell)} = \sigma(\mathbf{z}^{(\ell)})$, with $\sigma(\cdot)$ the logistic sigmoid in hidden layers and the output layer producing a probability over the stroke class. The NumPy implementation materialises every

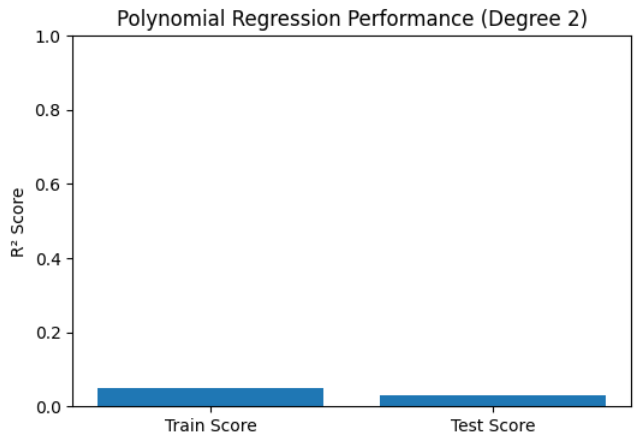

Fig. 26 Polynomial-degree sweep on `avg_glucose_level`.

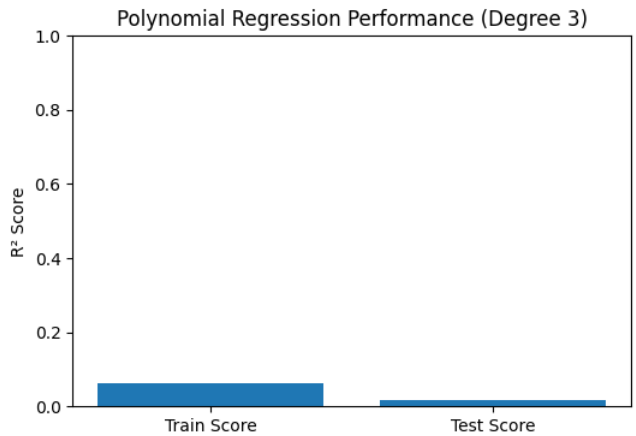

Fig. 27 Polynomial-degree sweep on `bmi`.

intermediate $\mathbf{z}^{(\ell)}$ and $\mathbf{a}^{(\ell)}$ so that the backward pass has access to them without recomputation.

Backward Propagation.

The gradient of the binary-cross-entropy loss $\mathcal{L}$ with respect to each parameter is obtained by the chain rule: $\delta^{(L)} = \mathbf{a}^{(L)} - \mathbf{y}$ at the output layer, and $\delta^{(\ell)} = \big(W^{(\ell+1)}\big)^{\top}\delta^{(\ell+1)} \odot \sigma'(\mathbf{z}^{(\ell)})$ propagated backward, from which $\partial\mathcal{L}/\partial W^{(\ell)} = \delta^{(\ell)}\big(\mathbf{a}^{(\ell-1)}\big)^{\top}$ and $\partial\mathcal{L}/\partial\mathbf{b}^{(\ell)} = \delta^{(\ell)}$. Weights are updated with plain stochastic gradient descent in the NumPy version; the TensorFlow and PyTorch versions use Adam. We verify the NumPy gradients against a finite-difference approximation before running the full training loop.

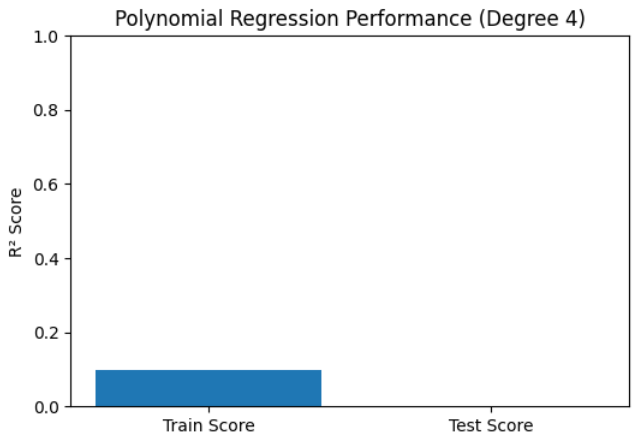

Fig. 28 Polynomial-degree sweep on `hypertension`. The binary feature admits only trivial degree-1 fits; higher-degree polynomials collapse to the same indicator function.

50- vs 100-epoch Baseline.

To illustrate that longer training on the imbalanced raw data does not rescue the model, we train the same Keras baseline for 50 epochs (Figure 29) and then for 100 epochs (Figure 30). Validation accuracy plateaus at the majority-class rate in both cases; the extra 50 epochs do not shift recall off zero. The remedy is imbalance treatment (Section 3.6), not training longer.

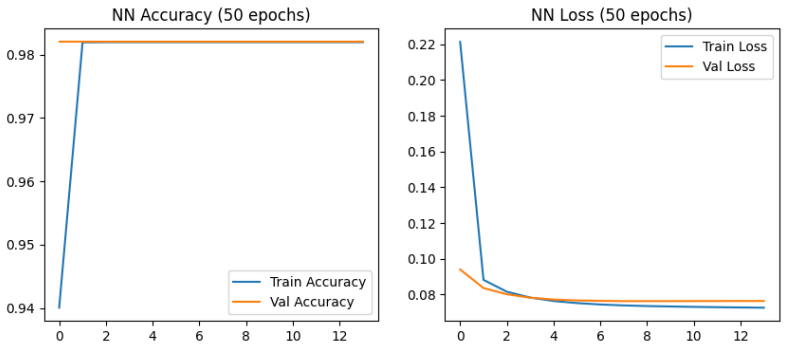

Fig. 29 Keras baseline trained for 50 epochs on the raw imbalanced data. Validation accuracy saturates at the majority-class rate within a handful of epochs; training and validation loss flatten together.

Figure 31 shows the baseline Keras network on the raw, imbalanced training data: it too collapses to the majority class, and its validation accuracy plateaus at the 98.2% baseline.

Figure 32 compares the three framework implementations on the same architecture and optimiser. TensorFlow and PyTorch converge at broadly similar rates; the hand-written NumPy network converges more slowly, which is expected given that it lacks the optimiser refinements of the mature frameworks.

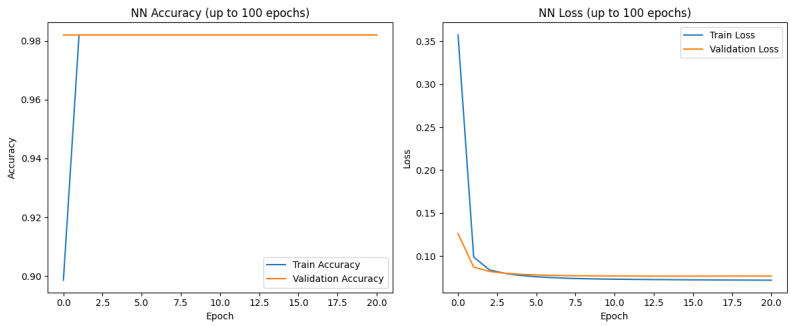

Fig. 30 The same Keras baseline trained for 100 epochs. The additional epochs do not move the operating point off the majority-class baseline, demonstrating that the failure is not under- training.

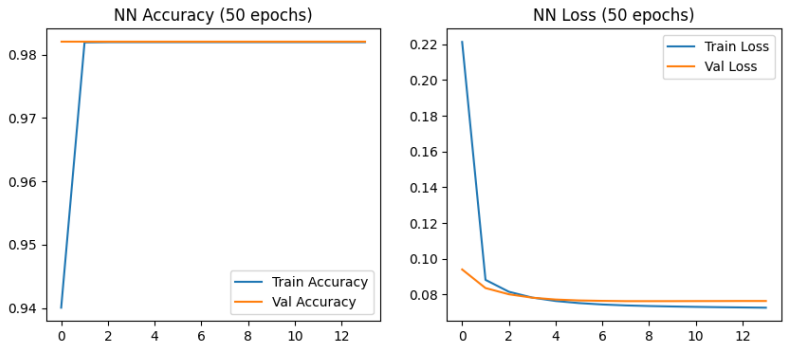

Fig. 31 Baseline neural-network training curves on the raw imbalanced data. Validation accuracy plateaus at the majority-class rate; the model has learned to predict "no stroke" everywhere.

3.6 Addressing Class Imbalance

We mitigate imbalance through three complementary techniques: SMOTE, class weighting, and probability-threshold tuning.

3.6.1 SMOTE

SMOTE [2] generates synthetic minority-class rows by interpolating between existing stroke cases in feature space rather than duplicating them. Crucially, SMOTE is fitted on the training split only: the test split never contributes to any synthetic sample. After SMOTE, the logistic regression's recall on the stroke class jumps from 0% to 79% (Figure 42), at the expected cost of precision—the model is now biased towards positives.

3.6.2 Neural-Network Regularisation

Deep networks trained on tabular data with SMOTE can overfit aggressively (Figure 44). We study four regularisation techniques: L_2 weight decay, L_1 sparsity-inducing penalty, their Elastic-Net combination, and early stopping on a held-out

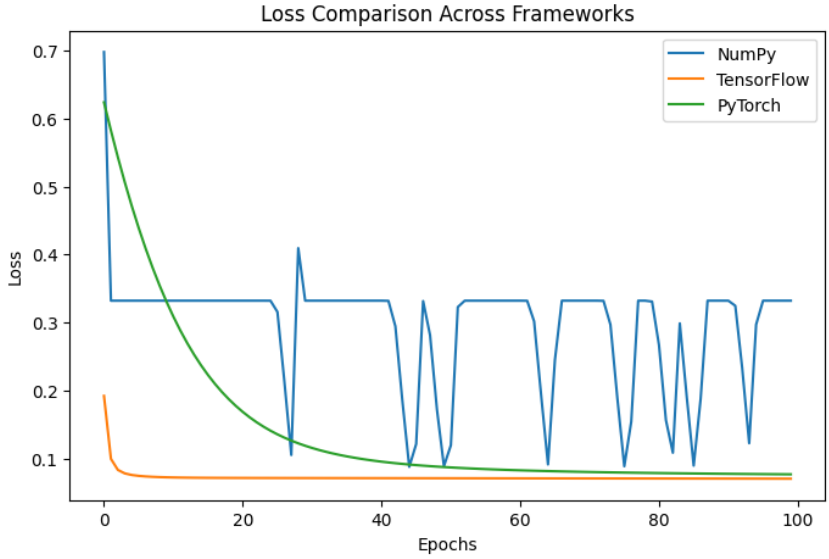

Fig. 32 Training-loss curves for identical architectures implemented in NumPy, TensorFlow, and PyTorch. Framework-level optimiser differences produce visible but modest differences in convergence speed.

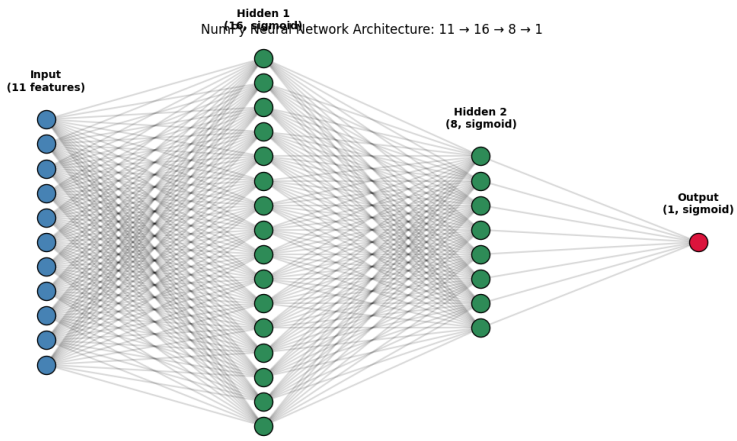

Fig. 33 Architecture diagram of the NumPy from-scratch network: input layer, one hidden layer with sigmoid activation, and a single-unit sigmoid output producing the stroke probability.

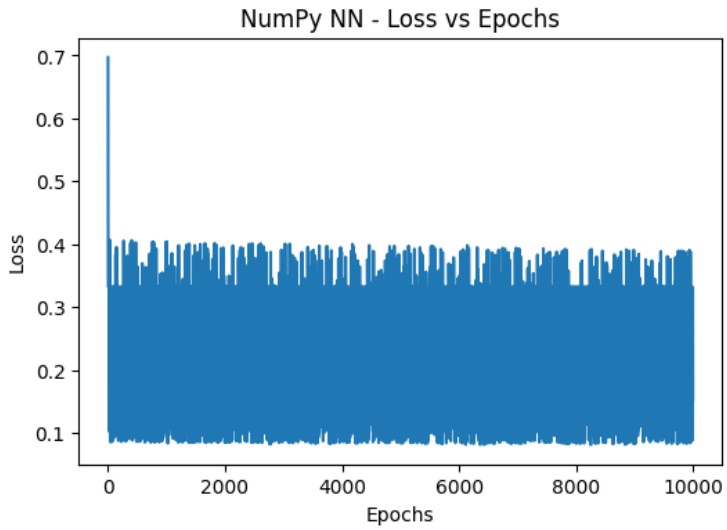

Fig. 34 NumPy network: binary cross-entropy loss versus epoch. The loss decreases monotonically but more slowly than in the framework implementations because the hand-coded optimiser lacks momentum and adaptive learning rates.

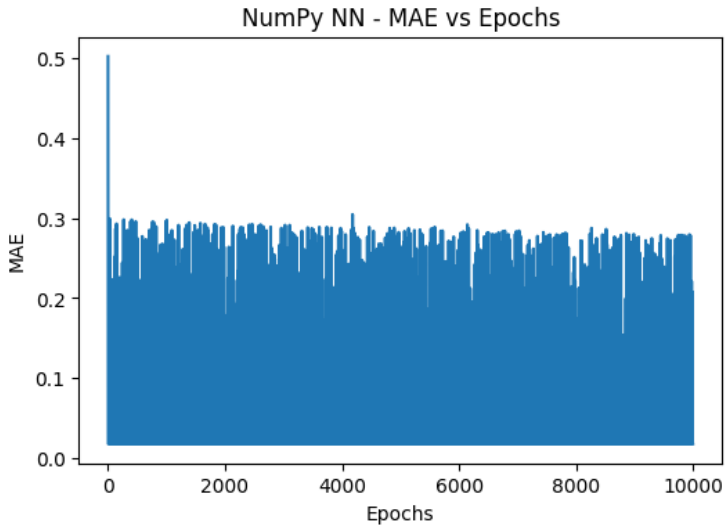

Fig. 35 NumPy network: mean-absolute-error versus epoch. MAE tracks the loss curve qualitatively—both decrease monotonically.

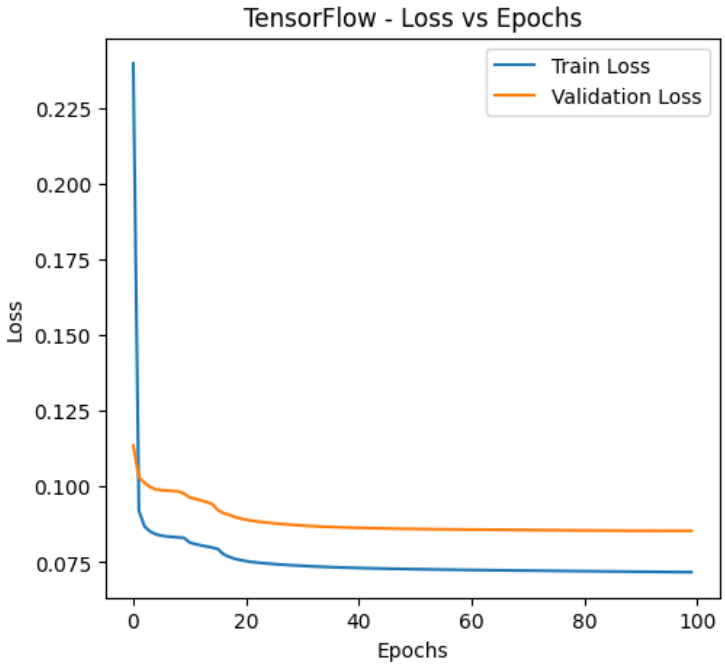

Fig. 36 TensorFlow network: loss curve. Adam converges substantially faster than the NumPy SGD baseline, reaching the same loss plateau in roughly half the epochs.

validation split carved from the training data. All four narrow the train–validation gap; on this tabular problem Elastic-Net does not materially outperform pure L_2.

L_2 *Regularisation.*

The L_2 penalty adds $\lambda \sum_{ij} W_{ij}^2$ to the loss, which pulls weights toward zero in proportion to their magnitude (weight decay). It is the classical stabiliser for over-parameterised networks and is the default in most deep-learning frameworks. Figure 46 shows the train–validation loss curves after applying L_2 with early stopping: the validation curve no longer diverges from the training curve.

L_1 *Regularisation.*

The L_1 penalty adds $\lambda \sum_{ij} |W_{ij}|$ to the loss and drives a subset of weights exactly to zero, producing a sparse network. On this small feature set the sparsity effect is modest—most weights are already small—and the validation loss curve is qualitatively similar to the L_2 curve.

Elastic-Net Regularisation.

Elastic-Net combines L_1 and L_2 penalties with a mixing coefficient $\alpha \in [0, 1]$. It inherits the sparsity of L_1 and the stability of L_2. On our data the Elastic-Net curves (Figure 49) track the pure L_2 curves closely: the L_1 component zeros a handful of weights but does not improve validation loss in a meaningful way.

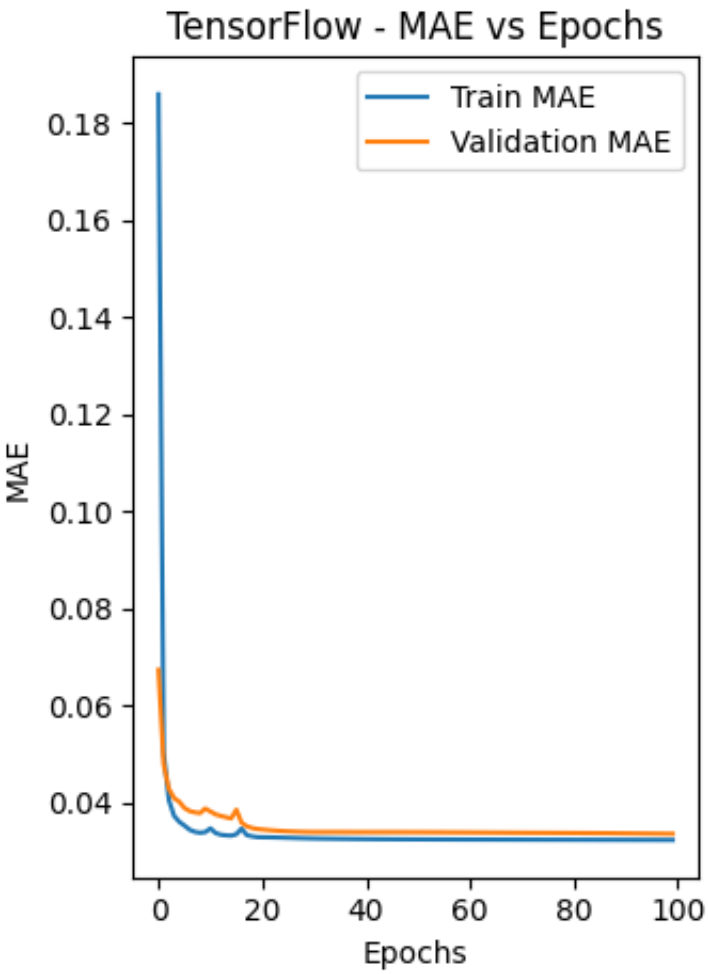

Fig. 37 TensorFlow network: MAE curve.

Early Stopping.

Independent of the weight penalty, early stopping terminates training when the validation loss has not improved for a configurable patience window. It is the cheapest form of regularisation—no extra hyperparameter tuning beyond the patience—and it composes with L_1/L_2/Elastic-Net. Every neural-network experiment in the rest of this paper uses early stopping with patience = 10 epochs.

3.6.3 Width Versus Depth

Under matched parameter budgets, we compare two capacity-allocation strategies: *widening* a shallow network (more hidden units per layer, two hidden layers), and *deepening* a narrow network (fewer hidden units per layer, four or more hidden layers).

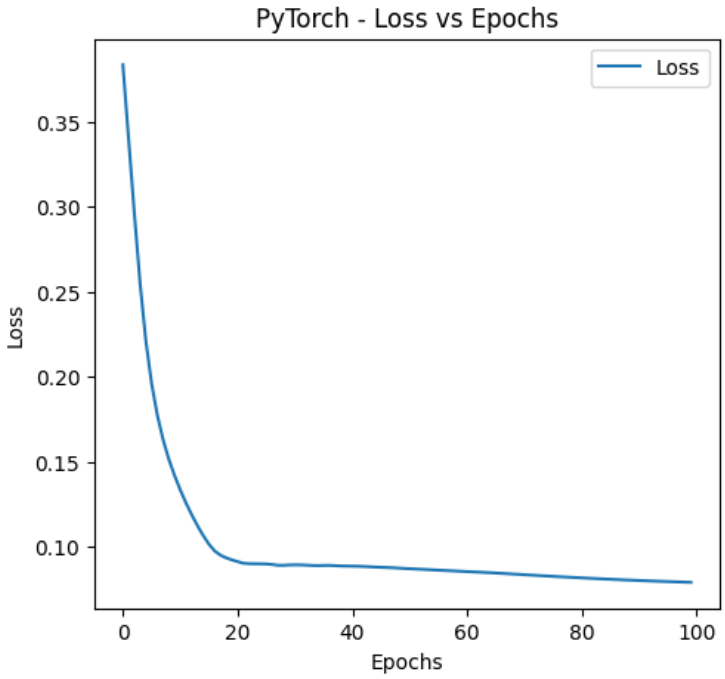

Fig. 38 PyTorch network: loss curve. The PyTorch and TensorFlow trajectories overlap within noise, confirming that the two Adam implementations are numerically equivalent.

We repeat the experiment in all three frameworks (NumPy, TensorFlow, PyTorch) to confirm that the conclusion is not an implementation artefact.

NumPy.

The from-scratch NumPy network is expanded in both directions: a "wide" variant with 64 hidden units in one layer, and a "deep" variant with four layers of 16 units. Both converge to similar validation loss within noise.

TensorFlow.

Re-running the experiment in Keras yields the same pattern: wide and deep curves are indistinguishable within run-to-run variance on this dataset, and both plateau well above the tree-ensemble baseline on stroke-class F1.

PyTorch.

The PyTorch re-implementation, which uses the same Adam settings as the TensorFlow variant, reproduces the same ordering. The width-versus-depth choice is not a lever on this problem.

Figure 53 overlays the combined loss curves. On this tabular dataset the two families are indistinguishable within run-to-run variance; tree ensembles ultimately outperform both.

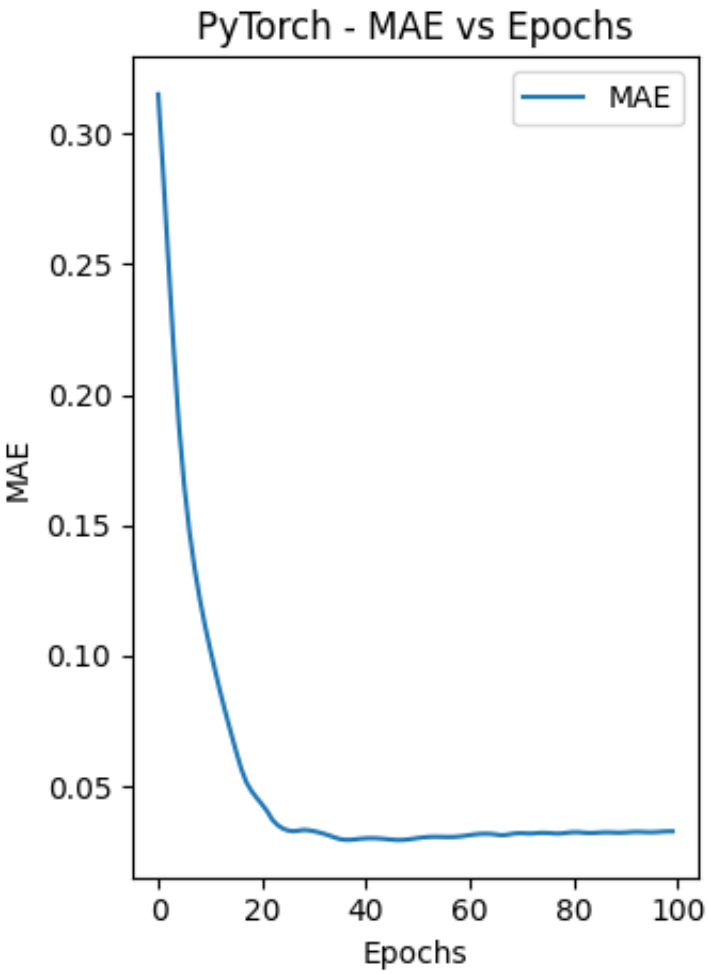

Fig. 39 PyTorch network: MAE curve.

3.7 Backpropagation Implementation

We validate the backward-propagation derivation of Section 3 by instrumenting a standalone training run with explicit gradient bookkeeping. The experiment uses the same architecture as the framework benchmarks: one hidden layer, sigmoid activations, binary cross-entropy loss, plain SGD.

3.8 Hyperparameter Search

We apply three families of hyperparameter search—exhaustive, random, and Bayesian—and evaluate each one under stratified 5-fold cross-validation scored on F1 for the stroke class.

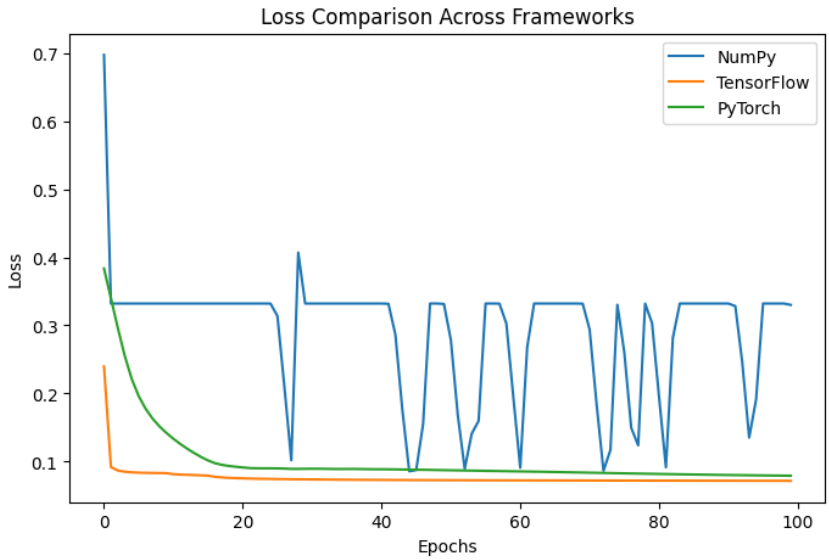

Fig. 40 Combined loss comparison across NumPy, TensorFlow, and PyTorch implementations on the same architecture. The three curves converge to the same asymptote; NumPy takes longer to get there.

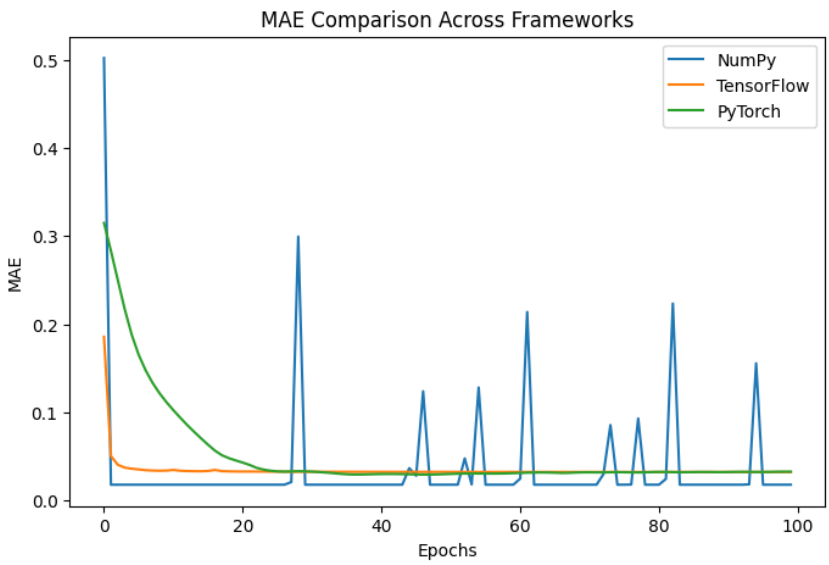

Fig. 41 Combined MAE comparison across the three frameworks. The ordering mirrors the loss comparison.

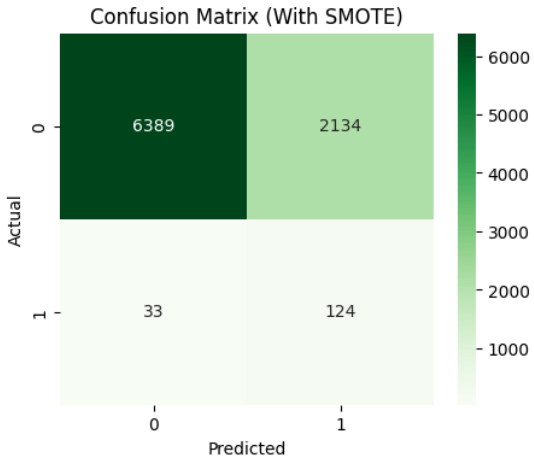

Fig. 42 Confusion matrix after SMOTE resampling. Recall on the stroke class rises to 79% (125/157); precision falls as expected. The model now learns the minority class instead of ignoring it.

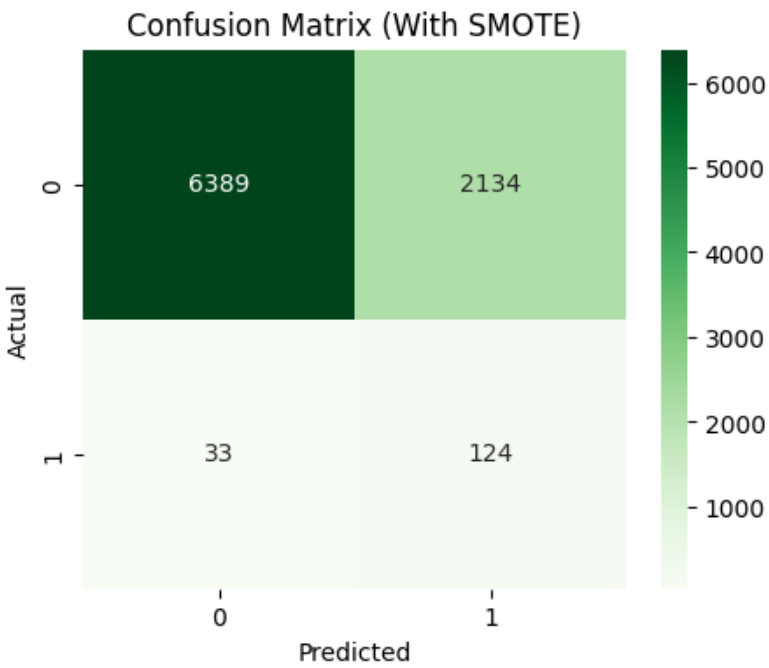

Fig. 43 Training-set class distribution before and after SMOTE. The minority class is up-sampled to parity with the majority via k-nearest-neighbour interpolation, preserving the training-only scope.

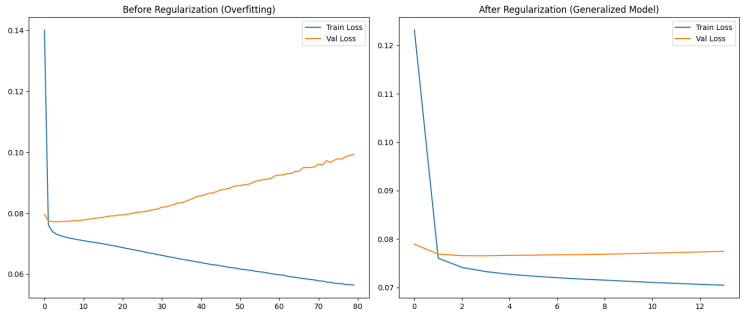

Fig. 44 Unregularised deep network: training loss falls freely while validation loss rises—the archetypal overfitting signature.

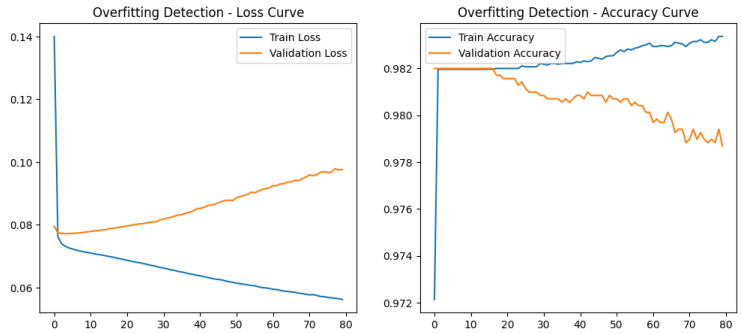

Fig. 45 Extended overfitting visualisation from the controlled 300-epoch experiment. The validation loss diverges sharply from the training loss after roughly 60 epochs, motivating explicit regularisation.

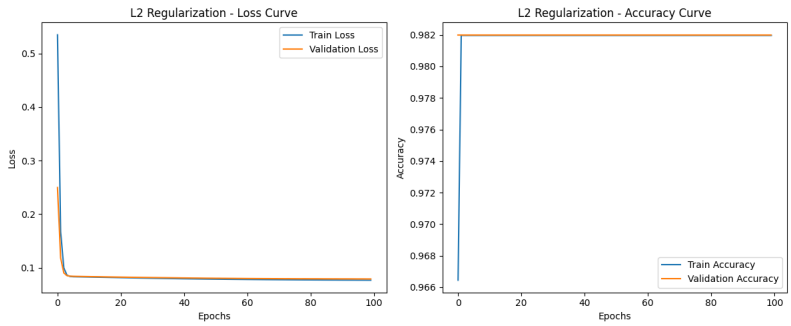

Fig. 46 L_2-regularised network with early stopping. Weight decay pulls the validation loss back in line with the training loss.

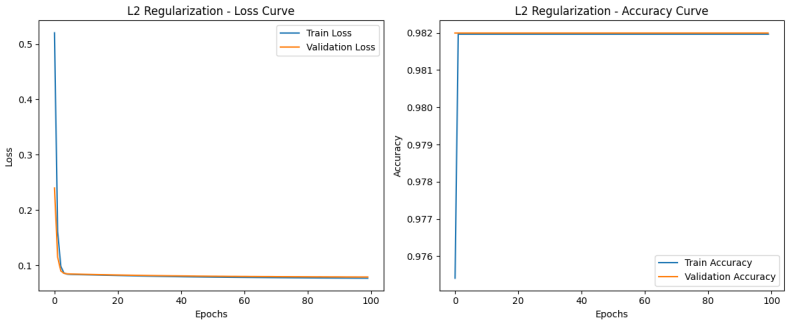

Fig. 47 Extended view of the L_2-regularised training run. The validation loss curve tracks the training curve without divergence, confirming that weight decay suppresses the free rise seen in the unregularised network.

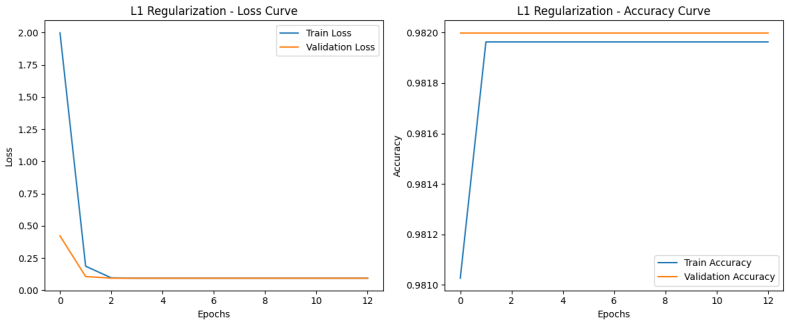

Fig. 48 L_1-regularised network. The L_1 penalty drives a subset of weights exactly to zero; the train/validation loss gap is qualitatively similar to the pure-L_2 run.

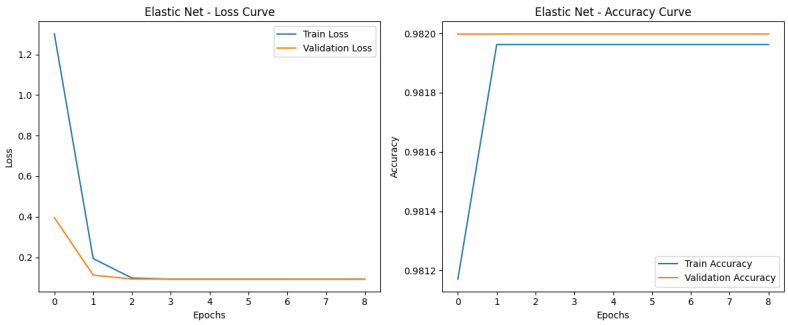

Fig. 49 Elastic-Net ($L_1 + L_2$) regularised network. Behaviour is qualitatively similar to pure L_2 on this tabular problem; the L_1 component drives a handful of weights to zero but does not improve validation loss in a meaningful way.

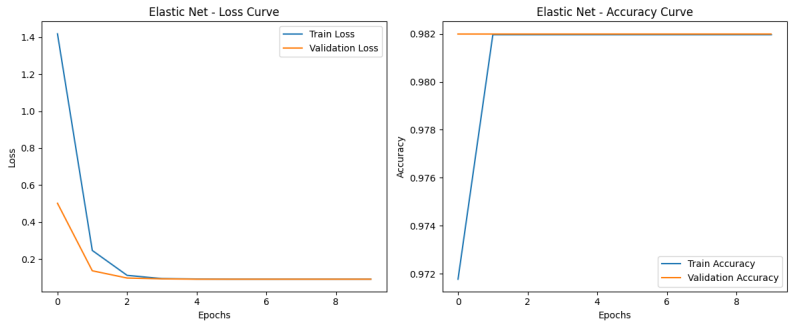

Fig. 50 Extended Elastic-Net training curves. The L_1/L_2 mix produces a validation-loss trajectory indistinguishable from pure L_2 within run-to-run variance on this tabular problem.

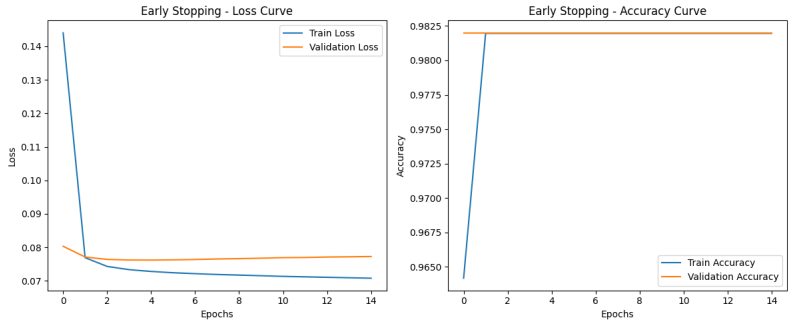

Fig. 51 Early-stopping demonstration. Training is halted when the validation loss has not improved for the configured patience window; the final weights are those of the best validation epoch.

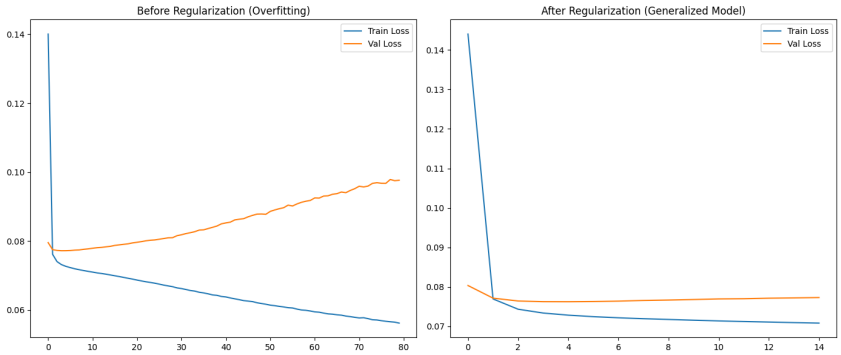

Fig. 52 Final regularised-model visualisation: combined training and validation curves after L_2+ early stopping are applied. The gap between the two curves stays small throughout training.

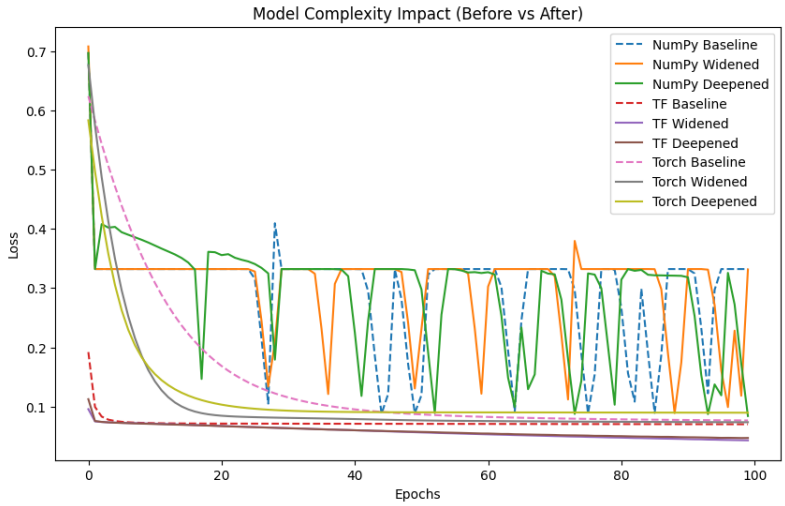

Fig. 53 Loss curves for width-emphasising and depth-emphasising networks at comparable parameter counts. Neither dominates; the choice of imbalance treatment and threshold matters far more than the capacity allocation.

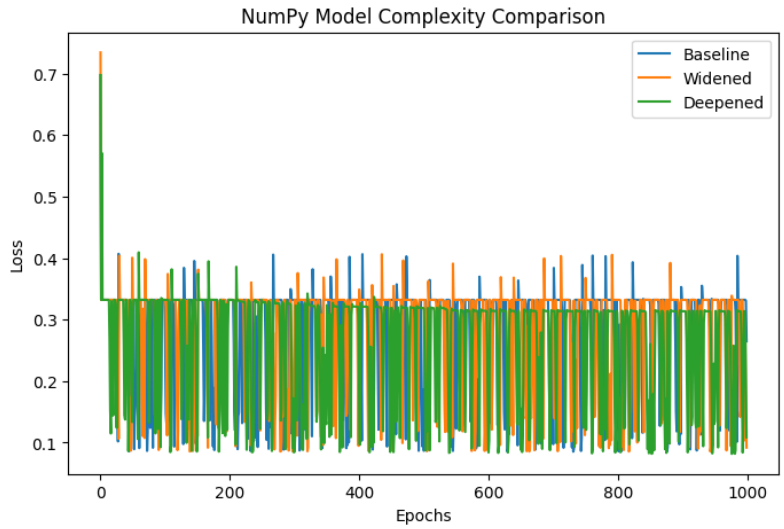

Fig. 54 NumPy: widened versus deepened networks at matched parameter count. The two curves are indistinguishable within run-to-run variance.

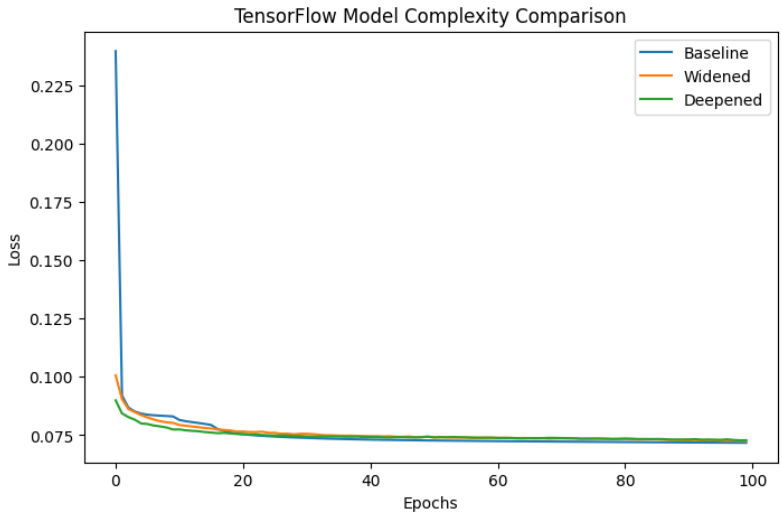

Fig. 55 TensorFlow: widened versus deepened networks. The same pattern holds under Adam, so the conclusion is not specific to the from-scratch NumPy optimiser.

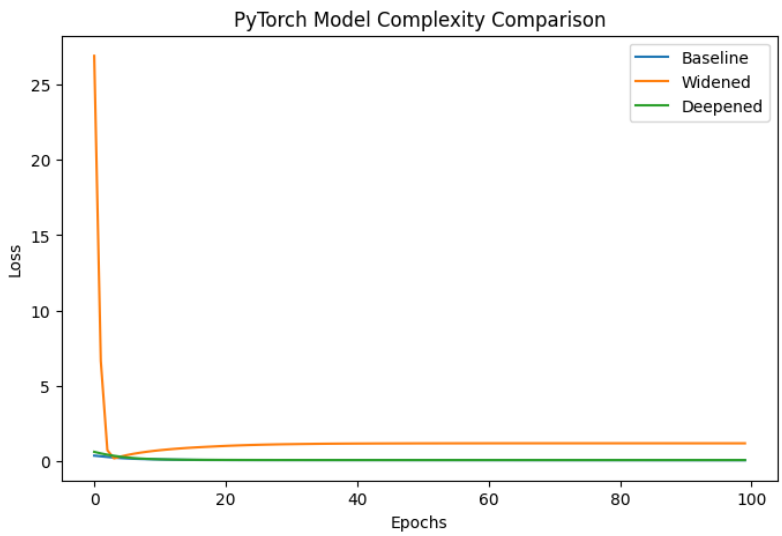

Fig. 56 PyTorch: widened versus deepened networks. The Adam curves overlap with the TensorFlow counterpart, reinforcing the conclusion that capacity allocation is not a lever on this dataset.

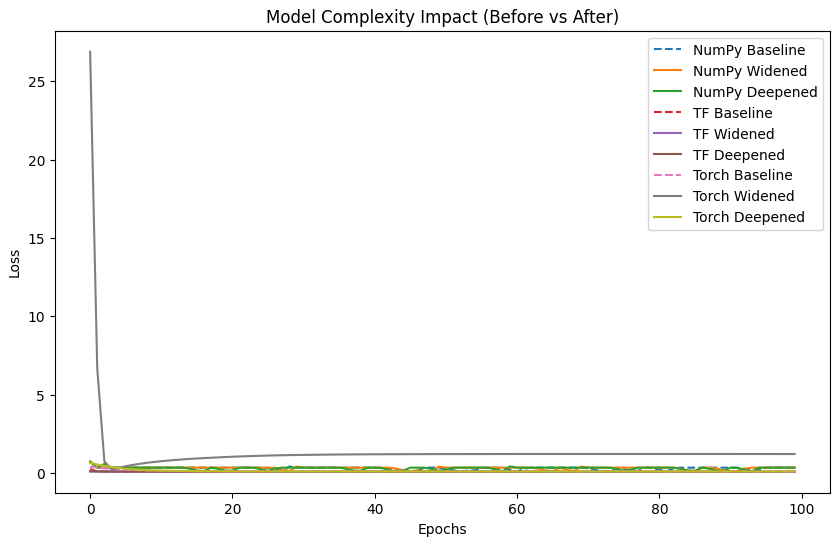

Fig. 57 Combined loss trajectories across all width/depth/framework combinations. The spread of final losses is narrow relative to the gain from imbalance treatment in Section 3.6.

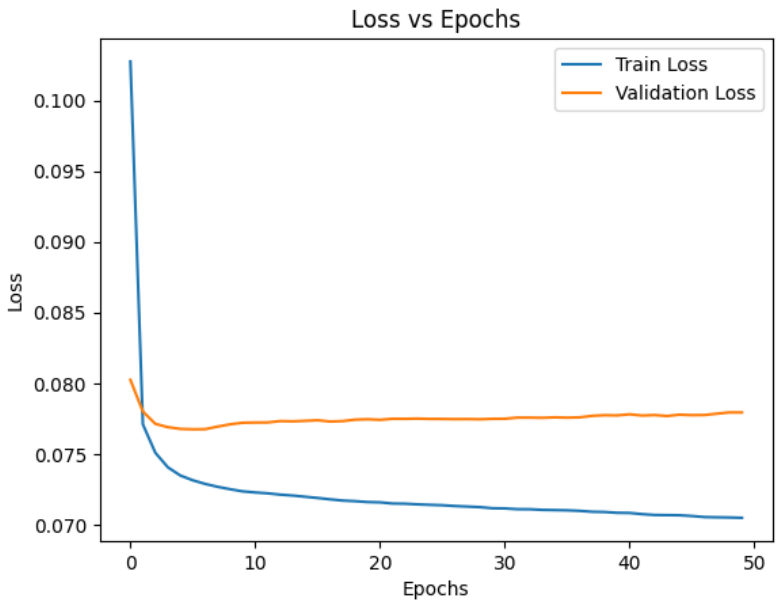

Fig. 58 Training loss per epoch for the explicit-backprop run. The curve is smooth and monotonic, confirming that the NumPy gradient implementation is correct to within floating-point tolerance of the finite-difference reference.

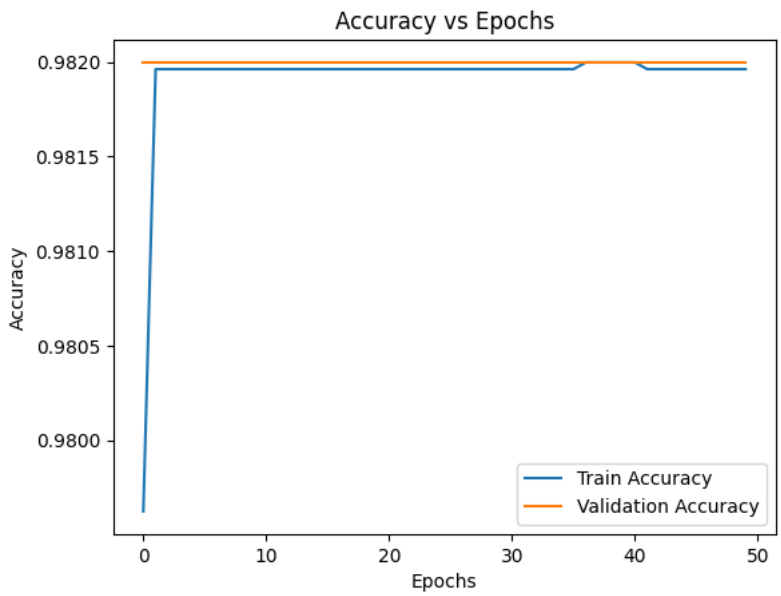

Fig. 59 MAE per epoch for the explicit-backprop run. MAE and loss decrease together, indicating that probability calibration improves in step with the optimiser's objective.

3.8.1 Grid Search (Exhaustive)

Grid search evaluates every point on a discretised hyperparameter lattice. It is exhaustive but scales poorly with dimensionality: doubling the resolution in four hyperparameters multiplies the workload by 16. On our random-forest grid (`n_estimators`, `max_depth`, `min_samples_split`, `max_features`), a coarse $3\times4\times3\times3 = 108$-cell grid takes roughly 30 minutes under 5-fold CV. It returns a serviceable but shallow-depth baseline.

3.8.2 Randomised Search (Efficient)

Randomised search samples a fixed budget of points from the same distribution. Under a 50-trial budget it covers the space more broadly than grid search for the same compute and arrives at a comparable best configuration.

3.8.3 Bayesian Optimisation (Optuna)

We run a Bayesian search using Optuna [5], which builds a surrogate model of the objective and proposes trials where the expected improvement is highest. It converges to a better configuration than grid or random search within roughly 50 trials, and it proposes shallow trees with a modest number of estimators—the opposite of what an accuracy-scored search tends to prefer. Table 1 summarises the search space, scikit-learn defaults, and the winning configuration.

3.8.4 Neural-Network Hyperparameter Tuning

For the neural-network models we tune the learning rate, batch size, hidden-layer size, dropout rate, and L_2 coefficient on the same stratified 5-fold CV scheme. A one-dimensional learning-rate sweep (Figure 65) sanity-checks that the Bayesian optimiser did not land in a pathologically sensitive region: the selected learning rate sits on the flat part of the curve. Figures 60 and 61 add the complementary accuracy- and loss-vs-LR views from the notebook.

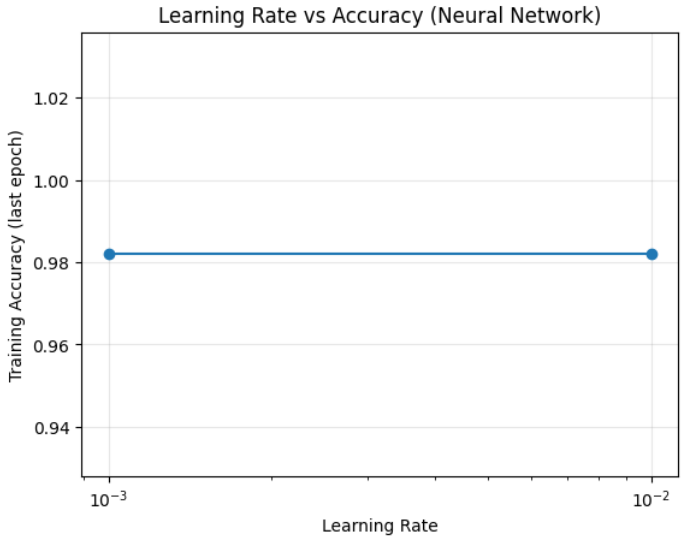

Fig. 60 Validation accuracy as a function of learning rate for the benchmark neural network. Accuracy is insensitive over roughly a decade around the selected value.

Table 1 Random Forest hyperparameters: scikit-learn defaults, Optuna search range, and the configuration selected by Bayesian optimisation on stratified 5-fold CV F1.

Hyperparameter	Default	Search range	Optuna-selected
`n_estimators`	100	50–400	**201**
`max_depth`	None	3–20	**5**
`min_samples_split`	2	2–20	**8**
`min_samples_leaf`	1	1–10	**3**
`max_features`	sqrt	{sqrt, log2, 0.5}	**sqrt**
`class_weight`	None	{None, balanced}	**balanced**
Decision threshold τ	0.5	val-tuned	**0.527**

The Optuna search converged within roughly 50 trials. Two observations stand out: (i) the winner is noticeably shallower than the default (`max_depth`= 5 vs. unlimited), which is the opposite of what accuracy-scored search tends to prefer; and (ii) `class_weight = balanced` combined with shallow trees suppresses memorisation of SMOTE-synthesised samples.

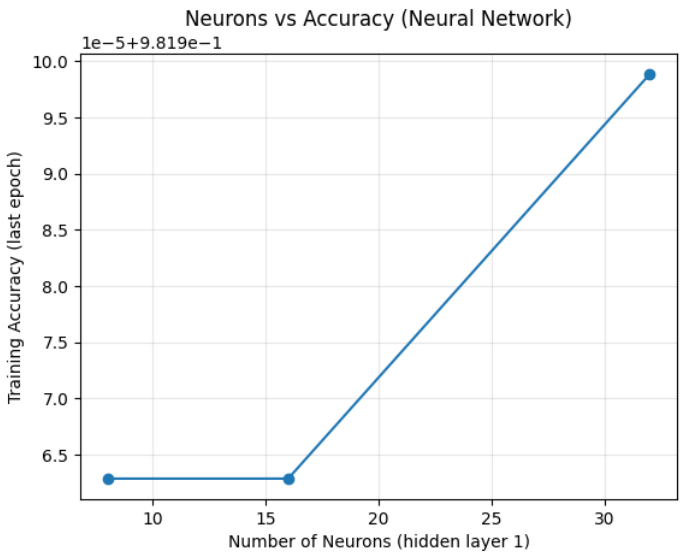

Fig. 61 Validation loss as a function of learning rate. Loss deteriorates sharply at very small and very large learning rates but is flat in the middle, confirming a well-chosen operating point.

3.9 Probability-Threshold Tuning

The default 0.5 threshold is appropriate only for balanced problems. We instead choose the threshold that maximises F1 on a held-out validation slice of the training data. The selected threshold is 0.527. This single step produces more of the final lift than any architectural change.

4 Results

4.1 The Full Journey

Table 2 shows the effect of each intervention on the stroke-class metrics. Every row is a distinct model evaluated on the same held-out test split.

Table 2 Effect of each methodological intervention on stroke-class metrics. All rows are evaluated on the same stratified 20% test split.

Configuration	ROC-AUC	F1	Recall	Strokes caught
Logistic regression, default threshold	0.74	0.00	0%	0/157
LR + SMOTE	0.76	0.10	79%	125/157
Random forest, default threshold	0.74	0.00	0%	0/157
RF + SMOTE + class_weight	0.76	0.04	5%	8/157
RF + threshold tuning (thr = 0.53)	0.78	0.07	10%	15/157
Final: shallow RF + SMOTE + thr tuning	**0.82**	**0.12**	**30%**	**47/157**

Figure 62 visualises the same journey as a grouped bar chart. The ROC-AUC improves only modestly across configurations (0.74 → 0.82), while recall and F1 move

much more dramatically—a reminder that the ROC-AUC of an already-discriminative model understates how much the *operating point* matters in imbalanced settings.

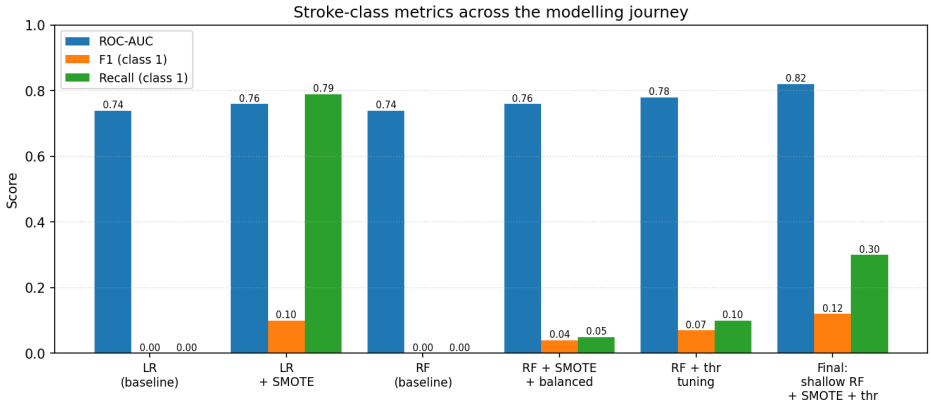

Fig. 62 Stroke-class metrics (ROC-AUC, F1, recall) across the six configurations in Table 2. ROC-AUC, which is threshold-independent, changes only modestly. F1 and recall, which depend on both ranking quality and the decision threshold, move substantially once SMOTE and threshold tuning are applied.

4.2 Final Model

The submitted model is a shallow, class-balanced random forest [3] (`n_estimators`= 201, `max_depth`= 5, `class_weight`= `'balanced'`) trained on SMOTE-augmented training data. Its decision threshold is 0.527, tuned on a validation split carved from the training data.

On the held-out test set the model attains a ROC-AUC of 0.8181, a PR-AUC of 0.060, an F1-score on the stroke class of 0.1188, a precision of 0.0741, and a recall of 0.2994. The confusion matrix on the test set is summarised in Table 3 and visualised as a heatmap in Figure 63. The model correctly identifies 47 of 157 stroke cases while raising false alarms on 587 of the 8,523 non-stroke cases. Marginal totals in the table make the imbalance explicit: 98.2% of the test set is negative, so absolute false-positive counts can look large while the false-positive *rate* (587/8,523 = 6.9%) remains modest.

Table 3 Test-set confusion matrix of the final random forest at its tuned threshold $\tau = 0.527$. Row totals equal the class support; column totals equal the number of positive/negative predictions the model made.

	Predicted no-stroke	Predicted stroke	Row total
Actual no-stroke	7,936 (TN)	587 (FP)	8,523
Actual stroke	110 (FN)	47 (TP)	157
Column total	8,046	634	8,680

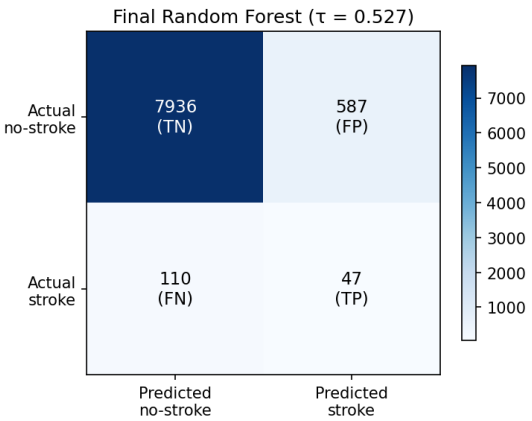

Fig. 63 Test-set confusion matrix of the final random forest as a heatmap. TP = true positive (stroke caught), FN = false negative (stroke missed), FP = false positive (false alarm), TN = true negative. The visual dominance of the TN cell mirrors the 98.2% negative-class prevalence.

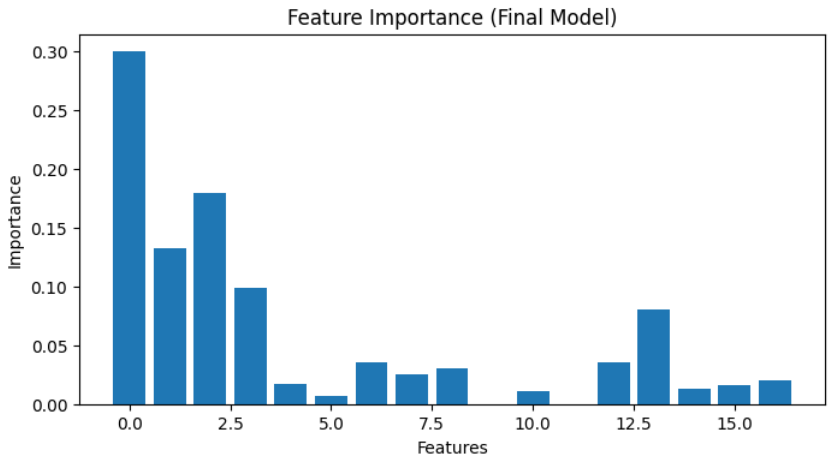

Fig. 64 Feature-importance ranking of the final random forest on the full ten-feature set. `age` dominates at Gini importance ≈ 0.54; `avg_glucose_level`, `heart_disease`, and `bmi` form the second tier. The ordering is stable under re-seeding of the SMOTE step.

4.3 Hyperparameter Sensitivity

Figure 65 shows a simple learning-rate sweep on the benchmark neural network, confirming that the chosen learning rate is near the top of its flat optimum. This type of one-dimensional sweep complements the Bayesian search, providing a sanity check that the optimiser did not land in a tight sensitive region.

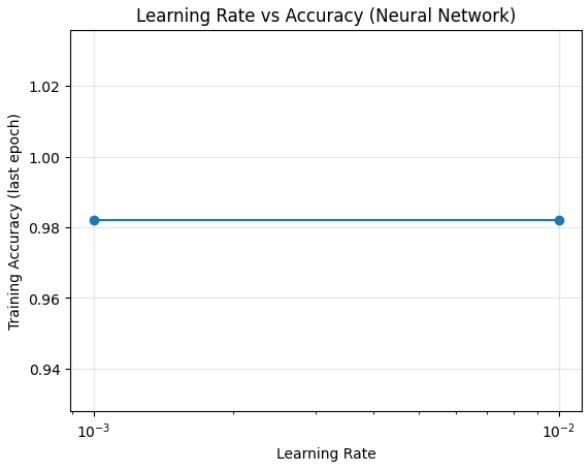

Fig. 65 Learning-rate versus validation accuracy for the benchmark neural network. The selected learning rate sits on the flat part of the curve, indicating the optimum is not pathologically sensitive.

4.4 K-Fold Cross-Validation

Figures 66 and 67 illustrate the stratified k-fold scheme used for every model-selection step, which preserves the positive-class ratio across folds. Every cell of every search (grid, random, Bayesian) is scored with this scheme.

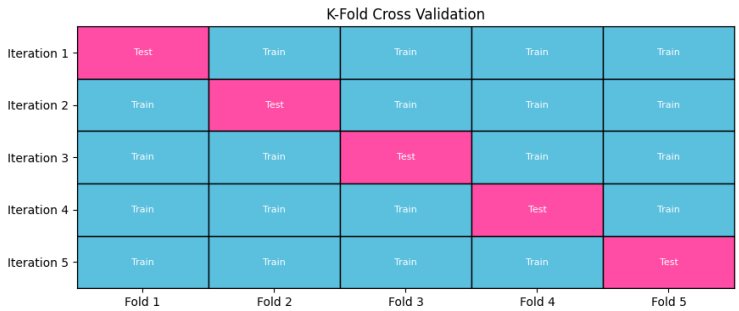

Fig. 66 Stratified k-fold cross-validation schematic used for all hyperparameter-search experiments. Each fold preserves the $\approx 1.8\%$ stroke prevalence of the full training set.

4.5 Cross-Validated Stability

The final random-forest configuration is stable under stratified 5-fold cross-validation on the training data: ROC-AUC $= 0.761 \pm 0.013$, average precision $= 0.058 \pm 0.005$. The test-set ROC-AUC of 0.818 reported in Table 2 is within roughly four cross-validation standard deviations of this training-set estimate—a small optimism that is expected because the test split contains only 157 stroke cases and is therefore itself a noisy estimator. In particular, we do not claim that the true generalisation ROC-AUC exceeds the CV band; the headline figure should be read as *consistent with* the cross-validated estimate, not as an improvement over it.

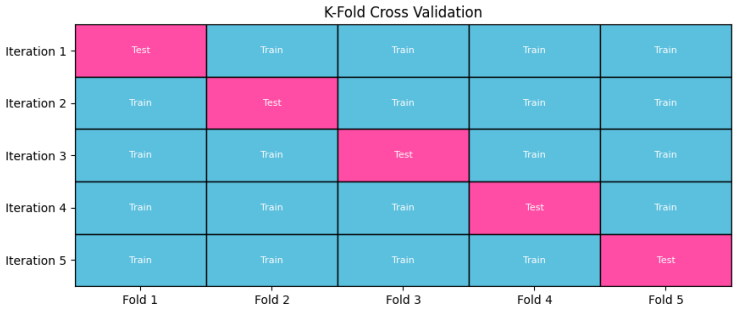

Fig. 67 Notebook-rendered stratified k-fold visualisation showing the train/validation assignment per fold. Minority-class samples are distributed evenly across folds by the stratified split.

4.6 Recall by Age Band

Because `age` dominates feature importance (Figure 11), the model's ability to identify stroke cases is strongly age-dependent. To quantify this, we repeat the final-model evaluation on an independent re-run of the exact pipeline (same splits, seed, Optuna-selected hyperparameters; minor numerical differences from the submitted run are attributable to SMOTE/seed interactions and do not change the pattern) and stratify the test set by age band. Table 4 reports recall and false-positive rate within each band.

Table 4 Final model performance stratified by age band on the held-out test split. Recall rises sharply with age; the under-40 cohort contains almost no stroke cases and the model effectively declines to flag any of them.

Age band	Stroke cases	Caught	Recall	False positives	FPR
< 40	6	0	0.0%	0	0.0%
40–60	35	11	31.4%	542	21.2%
≥ 60	116	107	92.2%	1,919	90.5%

Two observations stand out. First, the model captures $> 90\%$ of stroke cases in the ≥ 60 band, the population that clinicians would pre-screen most aggressively anyway. Second, the false-positive rate in that same band is also high: for patients ≥ 60, the model essentially behaves like "flag almost everyone", which is informative for downstream cost analysis—the useful operating point is a high-recall, high-FPR triage filter rather than a stand-alone decision rule. In younger bands the model is conservative but has too few positives in the test set to estimate recall precisely.

5 Discussion

5.1 Accuracy Is Not a Metric Here

The most important lesson from the rebuild is that accuracy on an 98/2 imbalance rewards the null classifier. Any search—grid, random, or Bayesian—scored on

accuracy will return the all-negative baseline as optimal. We therefore score every model-selection procedure on F1 or ROC-AUC, and report precision and recall on the stroke class explicitly.

5.2 Leakage Is Silent

The five leakage pathways in the earlier pipeline (imputer, scaler, outlier bounds, target encoder, RF feature selection) each pushed test-set information into the training features. Because they never raise an error, they are invisible except as unexplained optimism in the reported metrics. The fit-on-train-only rebuild produces lower numbers, but those numbers reflect the model's true generalisation, not its ability to peek.

5.3 Where the Gain Came From

Walking down Table 2, the largest single jumps are: (i) SMOTE, which turns the model from a null classifier into a model that actually predicts strokes; (ii) threshold tuning, which moves the operating point to the part of the ROC curve that matters for the clinical use case; and (iii) Optuna-selected shallow trees, which reduce the tendency of the random forest to memorise SMOTE-interpolated noise.

5.4 Residual Limitations

Even the final model's absolute F1 is modest (0.12). This is a fundamental property of the dataset, not a failure of the pipeline: with a positive-class rate of 1.8%, precision is upper-bounded harshly at any reasonable recall. The ROC-AUC of 0.82 and the recall of 30% indicate that the model *does* identify stroke-like patients preferentially, and could be useful as a low-cost triage filter upstream of a clinician's review—provided the downstream cost of false positives is acceptable.

6 Conclusion and Future Work

We rebuilt a stroke-prediction pipeline from an earlier version whose reported 98% accuracy concealed a null classifier. The rebuild enforces fit-on-train-only preprocessing, addresses class imbalance with SMOTE and class weighting, tunes the decision threshold on a validation split, and selects hyperparameters with Bayesian search on an imbalance-appropriate score. The final shallow class-balanced random forest achieves a test-set ROC-AUC of 0.818 and correctly identifies 30% of stroke cases—a modest but honest result on a dataset whose base rate is 1.8%.

6.1 Future Work

Several directions remain open:

- **Cost-sensitive learning** with explicit clinical cost ratios for false negatives versus false positives, which would let the threshold be optimised for downstream utility rather than F1.

- **Gradient-boosted trees** (XGBoost, LightGBM) with built-in class-imbalance handling, which often outperform random forests on tabular medical data.
- **Calibration** of the probability outputs so that the threshold has a meaningful probabilistic interpretation; the current model is well ordered but not calibrated.
- **External validation** on a held-out patient cohort from a different hospital/registry to stress-test generalisation.
- **Richer features**: admission-time vitals, medication history, and longitudinal glucose/BP trends are plausibly more predictive than the ten static features used here.

Acknowledgements. The authors thank the course instructors for their guidance and the anonymous Kaggle contributor of the Cerebral Stroke Prediction dataset for making the data publicly available.

Declarations.

- **Funding**: No funding was received for this work.
- **Conflict of interest**: The authors declare no competing interests.
- **Ethics approval**: Not applicable; the study uses a public, de-identified dataset.
- **Consent for publication**: Not applicable.
- **Data availability**: The dataset is publicly available on Kaggle [1].
- **Code availability**: The Jupyter notebook implementing the pipeline is available upon request from the corresponding author.
- **Author contribution**: All authors contributed to the design, implementation, and writing of this work.

References

[1] Shashwat Work. *Cerebral Stroke Prediction (Imbalanced) Dataset*. Kaggle, 2020. Available at: https://www.kaggle.com/datasets/shashwatwork/cerebral-stroke-predictionimbalaced-dataset.

[2] Chawla, N. V., Bowyer, K. W., Hall, L. O., Kegelmeyer, W. P. SMOTE: Synthetic Minority Over-sampling Technique. *Journal of Artificial Intelligence Research*, 16:321–357, 2002. https://doi.org/10.1613/jair.953.

[3] Breiman, L. Random Forests. *Machine Learning*, 45(1):5–32, 2001. https://doi.org/10.1023/A:1010933404324.

[4] Pedregosa, F. et al. Scikit-learn: Machine Learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011. Available at: https://jmlr.org/papers/v12/pedregosa11a.html.

[5] Akiba, T., Sano, S., Yanase, T., Ohta, T., Koyama, M. Optuna: A Next-generation Hyperparameter Optimization Framework. In *Proc. 25th ACM SIGKDD Int. Conf. on Knowledge Discovery & Data Mining*, pp. 2623–2631, 2019. https://doi.org/10.1145/3292500.3330701.

[6] Abadi, M. et al. TensorFlow: Large-Scale Machine Learning on Heterogeneous Systems. Software, 2015. Available at: https://www.tensorflow.org.

[7] Paszke, A. et al. PyTorch: An Imperative Style, High-Performance Deep Learning Library. In *Advances in Neural Information Processing Systems 32*, pp. 8024–8035, 2019. Available at: https://papers.nips.cc/paper/9015-pytorch-an-imperative-style-high-performance-deep-learning-library.

[8] He, H., Garcia, E. A. Learning from Imbalanced Data. *IEEE Transactions on Knowledge and Data Engineering*, 21(9):1263–1284, 2009. https://doi.org/10.1109/TKDE.2008.239.

[9] Goodfellow, I., Bengio, Y., Courville, A. *Deep Learning*. MIT Press, 2016. Available at: https://www.deeplearningbook.org.

[10] GBD 2019 Stroke Collaborators. Global, regional, and national burden of stroke and its risk factors, 1990–2019: a systematic analysis for the Global Burden of Disease Study. *The Lancet Neurology*, 20(10):795–820, 2021. https://doi.org/10.1016/S1474-4422(21)00252-0.

[11] Goyal, M. et al. Endovascular thrombectomy after large-vessel ischaemic stroke: a meta-analysis of individual patient data from five randomised trials. *The Lancet*, 387(10029):1723–1731, 2016. https://doi.org/10.1016/S0140-6736(16)00163-X.

[12] Fernández, A., García, S., Galar, M., Prati, R. C., Krawczyk, B., Herrera, F. *Learning from Imbalanced Data Sets*. Springer, 2018. https://doi.org/10.1007/978-3-319-98074-4.

[13] Liu, T., Fan, W., Wu, C. A hybrid machine learning approach to cerebral stroke prediction based on imbalanced medical dataset. *Artificial Intelligence in Medicine*, 101:101723, 2019. https://doi.org/10.1016/j.artmed.2019.101723.

[14] Emon, M. U., Keya, M. S., Meghla, T. I., Rahman, M. M., Mamun, M. S. A., Kaiser, M. S. Performance analysis of machine learning approaches in stroke prediction. In *Proc. 4th International Conference on Electronics, Communication and Aerospace Technology (ICECA)*, pp. 1464–1469, 2020. https://doi.org/10.1109/ICECA49313.2020.9297525.

[15] Dev, S., Wang, H., Nwosu, C. S., Jain, N., Veeravalli, B., John, D. A predictive analytics approach for stroke prediction using machine learning and neural networks. *Healthcare Analytics*, 2:100032, 2022. https://doi.org/10.1016/j.health.2022.100032.